

TRƯỜNG CAO ĐẲNG FPT POLYTECHNIC



FPT POLYTECHNIC

BÁO CÁO DỰ ÁN 1

Xây dựng Mô hình Phân loại nhị phân

Dự báo Rủi ro Khách hàng Vay vốn

Giáo viên hướng dẫn: NGUYỄN VĂN LONG

Sinh viên thực hiện:

Nhóm: 01, Lớp: AI21301

Thành viên:

- Nguyễn Thành Hưng, PS47270, Trưởng nhóm (Data Analyst)
- Nguyễn Bá Gia Huy, PS48224, ML Engineer
- Đỗ Trọng Thái, PS47694, Data Engineer
- Đặng Trịnh Qui Anh, PS48165, Data Engineer
- Phạm Đức Thắng, PS48172, Tester

TP Hồ Chí Minh – 2026

MỤC LỤC

MỤC LỤC.....	1
LỜI MỞ ĐẦU	3
Chương 1. Phân tích ý tưởng và bài toán nghiên cứu.....	6
1.1. Bối cảnh dự án.....	6
1.2. Phân tích mô hình SWOT	7
1.3. Mục tiêu nghiên cứu.....	8
1.4. Giới thiệu dataset.....	9
1.5 Giới thiệu công nghệ.....	12
1.6 Phương pháp và quy trình thực hiện	13
Chương 2. Xây dựng pipeline và tiền xử lý dữ liệu.....	14
2.1. Quy trình tích hợp dữ liệu	14
2.2. Kích bản kiểm thử dữ liệu sau khi Join	21
2.3. Làm sạch dữ liệu chuyên sâu	22
Chương 3. Phân tích sâu dữ liệu và tìm ý tưởng tạo feature.....	26
3.1. Phân tích khám phá dữ liệu (EDA).....	26
3.2. Xây dựng biến phái sinh (Feature Engineering)	34

Chương 4. Mô hình hóa và dự báo.....	43
4.1. Xây dựng mô hình dự báo rủi ro tín dụng.....	43
4.2. Đánh giá hiệu suất mô hình.....	44
4.3. Quá trình kiểm chứng và cải tiến feature sau đánh giá mô hình	47
4.4. Phân tích kết quả dự báo và gợi ý hành động	49
Chương 5. Quản trị dự án và phối hợp nhóm	54
5.1. Phân chia vai trò và nhiệm vụ (Team Roles & Responsibilities)	54
5.2. Kế hoạch và tiến độ triển khai	55
5.3. Quản lý mã nguồn và phiên bản trên Git/GitHub	57
5.4. Quy tắc cộng tác và đạo đức nghề nghiệp	59
Chương 6. Tổng kết và triển khai dự án	60
6.1. Hướng dẫn đóng gói và chạy dự án	60
6.2. Kết luận	63
6.3. Hướng phát triển	67

LỜI MỞ ĐẦU

Trong bối cảnh chuyển đổi số diễn ra mạnh mẽ, dữ liệu ngày càng trở thành nguồn lực quan trọng giúp doanh nghiệp đưa ra quyết định nhanh chóng và chính xác hơn. Đối với lĩnh vực tài chính tiêu dùng, việc đánh giá khả năng trả nợ của khách hàng có ý nghĩa đặc biệt quan trọng, bởi quyết định cho vay cần cân bằng giữa mục tiêu mở rộng cơ hội tiếp cận vốn và yêu cầu kiểm soát rủi ro tín dụng. Khi số lượng hồ sơ vay lớn và thông tin khách hàng đến từ nhiều nguồn khác nhau, việc phân tích thủ công khó bảo đảm tính nhất quán, kịp thời và khách quan. Vì vậy, ứng dụng trí tuệ nhân tạo và học máy vào dự báo rủi ro tín dụng là một hướng tiếp cận thiết thực, có giá trị cả về mặt học thuật lẫn thực tiễn.

Xuất phát từ nhu cầu đó, nhóm thực hiện đề tài “Xây dựng mô hình phân loại và dự báo rủi ro khách hàng vay vốn” trên bộ dữ liệu Home Credit Default Risk. Dự án xây dựng một quy trình xử lý dữ liệu từ nhiều bảng quan hệ, bao gồm tổ chức dữ liệu bằng PostgreSQL, làm sạch dữ liệu, phân tích khám phá, xây dựng đặc trưng và huấn luyện các mô hình phân loại. Kết quả hướng đến việc ước lượng xác suất khách hàng gặp khó khăn trong thanh toán, từ đó hỗ trợ nhận diện sớm các hồ sơ có rủi ro cao. Thông qua đề tài, nhóm mong muốn vận dụng kiến thức về cơ sở dữ liệu, phân tích dữ liệu và machine learning để tạo ra một sản phẩm AI có khả năng ứng dụng, đồng thời rèn luyện tư duy giải quyết bài toán nghiệp vụ dựa trên dữ liệu.

NHẬN XÉT CỦA GIẢNG VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Giáo viên hướng dẫn ký, ghi rõ họ tên

NHẬN XÉT CỦA HỘI ĐỒNG PHẢN BIỆN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

HĐ phản biện ký, ghi rõ họ tên

Chương 1. Phân tích ý tưởng và bài toán nghiên cứu

1.1. Bối cảnh dự án

1.1.1. Lý do lựa chọn bộ dữ liệu Home Credit Default Risk

Phù hợp với bài toán rủi ro tín dụng: Bộ dữ liệu cung cấp hồ sơ vay của khách hàng cùng biến mục tiêu TARGET, cho phép xây dựng và đánh giá mô hình phân loại khả năng khách hàng gặp khó khăn trong thanh toán.

Có quy mô lớn và cấu trúc đa bảng: Ngoài bảng hồ sơ chính, dữ liệu còn chứa lịch sử tín dụng tại tổ chức khác, hồ sơ vay trước đó, lịch sử trả góp, giao dịch POS/CASH và số dư thẻ tín dụng.

Phản ánh các vấn đề dữ liệu thực tế: Dữ liệu tồn tại giá trị thiếu, ngoại lệ, giá trị không hợp lệ và mất cân bằng lớp, phù hợp để thực hành các bước kiểm tra và xử lý dữ liệu có căn cứ nghiệp vụ.

Cho phép xây dựng pipeline AI hoàn chỉnh: Nhóm có thể tổ chức dữ liệu trong PostgreSQL, tổng hợp quan hệ một-nhiều trước khi join, tạo đặc trưng, huấn luyện, đánh giá và tối ưu ngưỡng dự đoán.

1.1.2. Tầm quan trọng của việc giải quyết bài toán trong ngành AI

Hỗ trợ nhận diện sớm rủi ro: Mô hình AI có thể phát hiện các hồ sơ có dấu hiệu khó khăn trong thanh toán để tổ chức tín dụng xem xét kỹ hơn.

Hỗ trợ phân bổ nguồn lực thẩm định: Kết quả dự báo giúp ưu tiên các hồ sơ cần kiểm tra chuyên sâu và góp phần xây dựng chính sách tín dụng phù hợp.

Khai thác dữ liệu lớn từ nhiều nguồn: Bài toán minh họa khả năng của AI trong việc tổng hợp dữ liệu hồ sơ, lịch sử tín dụng và hành vi thanh toán thành thông tin hỗ trợ quyết định.

Đặt ra yêu cầu sử dụng AI có trách nhiệm: Mô hình cần được đánh giá bằng metric phù hợp, giải thích rõ giới hạn và chỉ hỗ trợ sàng lọc; không nên được dùng để tự động từ chối hồ sơ vay.

1.1.3. Vai trò của mô hình trong quy trình tín dụng

Mô hình trong dự án được định vị là công cụ hỗ trợ chuyên viên tín dụng trong quá trình thẩm định hồ sơ, không thay thế hoàn toàn quyết định của con người. Thay vì chỉ đưa ra kết quả “chấp nhận” hoặc “từ chối”, mô hình ước lượng xác suất khách hàng gặp khó khăn trong thanh toán để hỗ trợ phân tầng mức độ rủi ro và ưu tiên kiểm tra các hồ sơ cần chú ý.

Việc định vị mô hình theo hướng hỗ trợ quyết định cũng ảnh hưởng trực tiếp đến cách xây dựng hệ thống. Mô hình cần trả về xác suất rủi ro, các chỉ số đánh giá phải phản ánh được khả năng phân biệt và phát hiện lớp khách hàng rủi ro, đồng thời ngưỡng phân loại cần được lựa chọn dựa trên mục tiêu nghiệp vụ thay vì mặc định ở mức 0,5.

1.2. Phân tích mô hình SWOT

1.2.1. Điểm mạnh

Dự án sử dụng bộ dữ liệu Home Credit Default Risk có quy mô lớn, gồm bảng hồ sơ vay chính và nhiều bảng lịch sử liên quan đến tín dụng, khoản vay trước đó, trả góp, POS/CASH và thẻ tín dụng. Sự đa dạng về biến số giúp nhóm có thể phân tích rủi ro dưới nhiều góc độ như nhân khẩu học, năng lực tài chính, tài sản, lịch sử thanh toán và hành vi tín dụng. Biến mục tiêu TARGET được xác định rõ ràng, tạo điều kiện thuận lợi để xây dựng và đánh giá các mô hình phân loại nhị phân.

1.2.2. Điểm yếu

Dữ liệu chứa nhiều giá trị thiếu, đặc biệt ở các nhóm thông tin lịch sử tín dụng và đặc điểm nhà ở. Một số biến có phân phối lệch, giá trị ngoại lệ hoặc giá trị không hợp lệ về mặt nghiệp vụ, chẳng hạn mã ngày làm việc đặc biệt và một số giá trị dư nợ âm. Ngoài ra, tỷ lệ khách

hàng gặp khó khăn trong thanh toán thấp hơn đáng kể so với nhóm còn lại, khiến dữ liệu bị mất cân bằng lớp và làm việc đánh giá mô hình trở nên phức tạp hơn.

1.2.3. Cơ hội

Dự án có cơ hội tìm ra các quy luật liên quan đến rủi ro tín dụng, chẳng hạn ảnh hưởng của thu nhập, tỷ lệ khoản vay trên giá trị tài sản, lịch sử chậm trả, dư nợ còn lại và điểm đánh giá bên ngoài đến khả năng vỡ nợ. Những insight này có thể hỗ trợ tổ chức tài chính nhận diện sớm nhóm khách hàng cần được kiểm tra kỹ hơn. Đồng thời, dự án là nền tảng để phát triển một ứng dụng dự báo rủi ro, cho phép người dùng nhập thông tin cơ bản và nhận kết quả ước lượng mức độ rủi ro.

1.2.4. Thách thức

Thách thức lớn của dự án là xử lý hiệu năng và bảo đảm tính chính xác khi kết hợp nhiều bảng dữ liệu lớn. Các bảng phụ thường có quan hệ một-nhiều với khách hàng; nếu join trực tiếp có thể làm nhân bản dòng dữ liệu, gây sai lệch thống kê và ảnh hưởng đến kết quả mô hình. Vì vậy, nhóm cần tổng hợp từng bảng phụ về mức một dòng trên mỗi khách hàng trước khi join với bảng chính, đồng thời sử dụng PostgreSQL, chỉ mục trên các khóa nối và các bước kiểm thử sau join để bảo đảm dữ liệu không bị mất hoặc sai lệch.

Đây là dạng lỗi đặc biệt nguy hiểm vì quá trình xử lý vẫn có thể chạy bình thường nhưng dữ liệu đầu ra đã bị sai lệch. Vì vậy, nhóm cần kiểm tra các bất biến như số lượng hồ sơ, tính duy nhất của SK_ID_CURR và phân bố TARGET sau quá trình tích hợp

1.3. Mục tiêu nghiên cứu

Mục tiêu của dự án là xây dựng một pipeline AI nhằm ước lượng xác suất khách hàng gặp khó khăn trong thanh toán, từ đó cung cấp thông tin hỗ trợ quá trình đánh giá và phân loại mức độ rủi ro tín dụng.

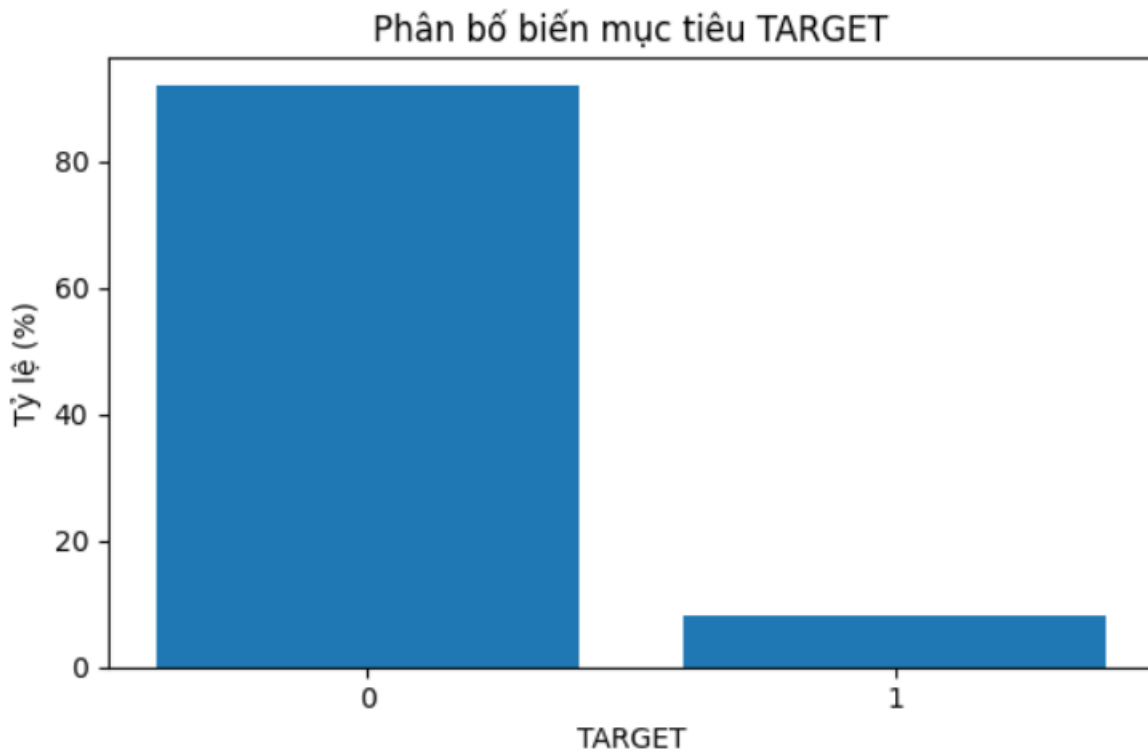
Dự án tập trung trả lời bốn câu hỏi nghiên cứu chính:

1. Những thông tin nào trong hồ sơ vay hiện tại có liên quan đến khả năng khách hàng gặp khó khăn trong thanh toán?
2. Lịch sử tín dụng, các khoản vay trước đây và hành vi thanh toán có giúp nhận diện rủi ro tốt hơn hay không?
3. Có thể xây dựng những đặc trưng nào phản ánh tốt hơn năng lực tài chính và hành vi tín dụng của khách hàng?
4. Mô hình học máy có thể hỗ trợ phân nhóm rủi ro tốt hơn các phương pháp đánh giá đơn giản hay không?

1.4. Giới thiệu dataset

Dự án sử dụng bộ dữ liệu Home Credit Default Risk, được cung cấp trên Kaggle, nhằm phục vụ bài toán dự báo khả năng khách hàng gặp khó khăn trong việc thanh toán khoản vay. Bảng dữ liệu trung tâm là applications, gồm 307.511 hồ sơ vay và biến mục tiêu TARGET. Trong đó, TARGET = 1 biểu thị khách hàng gặp khó khăn trong thanh toán, còn TARGET = 0 biểu thị khách hàng không gặp khó khăn theo định nghĩa của bộ dữ liệu.

Một đặc điểm quan trọng của dữ liệu là sự mất cân bằng giữa hai lớp. Trong dữ liệu ban đầu, TARGET = 1 chỉ chiếm khoảng 8,07% số hồ sơ, trong khi phần lớn khách hàng thuộc lớp TARGET = 0. Vì vậy, Accuracy không phù hợp để sử dụng làm chỉ số đánh giá chính; dự án ưu tiên các chỉ số như ROC-AUC, PR-AUC, Recall và F1-score để đánh giá mô hình.

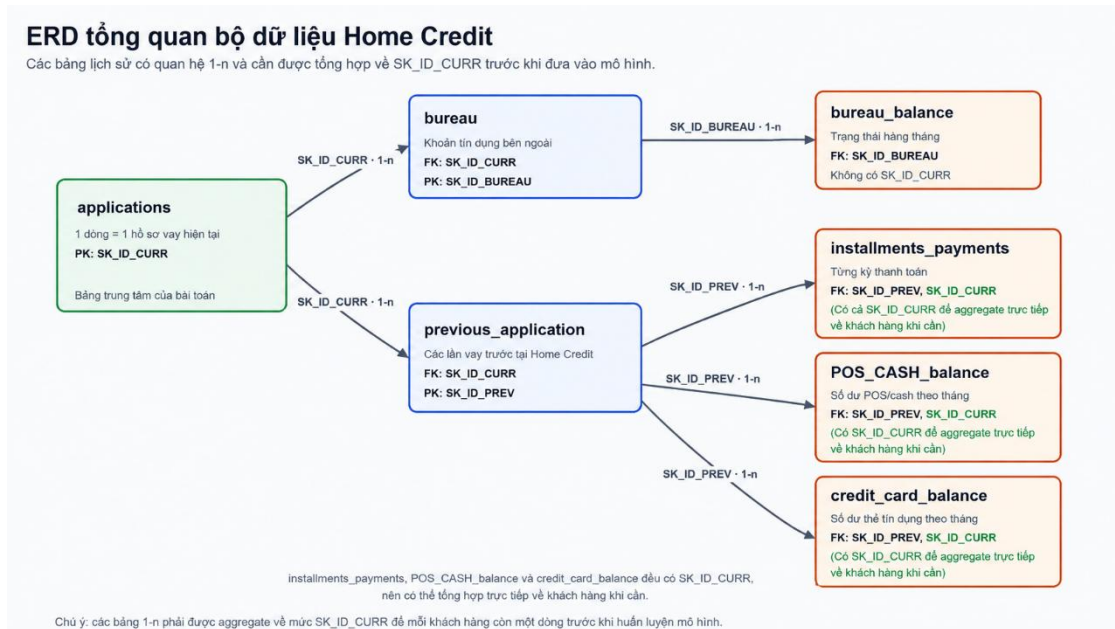


Hình 1.1 Phân bố biến mục tiêu TARGET trong bảng applications

Bên cạnh bảng hồ sơ chính, bộ dữ liệu gồm các bảng phụ phản ánh lịch sử tín dụng và hành vi tài chính của khách hàng. `bureau.csv` và `bureau_balance.csv` lưu thông tin tín dụng tại các tổ chức khác; `previous_application.csv` lưu các hồ sơ vay trước đó tại Home Credit; `installments_payments.csv` thể hiện lịch sử thanh toán trả góp; `POS_CASH_balance.csv` mô tả dư nợ POS/CASH; và `credit_card_balance.csv` lưu thông tin liên quan đến thẻ tín dụng. Các bảng này được liên kết chủ yếu qua mã khách hàng `SK_ID_CURR`, trong khi một số quan hệ cần sử dụng thêm mã khoản vay hoặc mã tín dụng như `SK_ID_PREV` và `SK_ID_BUREAU`.

Đặc điểm đáng chú ý của dataset là dữ liệu có nhiều quan hệ một-nhiều. Một khách hàng có thể có nhiều khoản vay cũ, nhiều kỳ thanh toán hoặc nhiều bản ghi lịch sử tín dụng. Vì vậy, nhóm không join trực tiếp toàn bộ bảng phụ vào bảng chính mà tổng hợp dữ liệu của từng khách hàng trước khi kết hợp. Cách xử lý này giúp bảo đảm mỗi khách hàng chỉ có một dòng dữ liệu, tránh làm sai lệch số lượng hồ sơ và tạo ra bộ dữ liệu phù hợp cho các bước làm sạch, phân tích và xây dựng mô hình dự báo.

Do các bảng lịch sử có quan hệ một-nhiều với khách hàng, việc join trực tiếp vào bảng applications có thể làm nhân bản hồ sơ. Vì vậy, các bảng phụ được tổng hợp về đúng grain trước khi kết hợp với bảng chính.



Hình 1.2 Sơ đồ quan hệ các bảng trong bộ dữ liệu Home Credit Default Risk

Sơ đồ cho thấy applications là bảng trung tâm; bureau_balance liên kết với bureau qua SK_ID_BUREAU, trong khi installments_payments, POS_CASH_balance và credit_card_balance liên kết với previous_application qua SK_ID_PREV. Các bảng phụ có quan hệ một-nhiều nên cần được tổng hợp trước khi tích hợp vào bảng khách hàng.

1.5 Giới thiệu công nghệ

Công nghệ	Vai trò	Lý do lựa chọn
PostgreSQL	Lưu trữ, tổng hợp và join dữ liệu	Phù hợp với dữ liệu nhiều bảng, quy mô lớn; hỗ trợ index và aggregation hiệu quả
Python, pandas, NumPy	Làm sạch và tạo đặc trưng	Hệ sinh thái xử lý dữ liệu linh hoạt
Matplotlib, seaborn	EDA và trực quan hóa	Hỗ trợ phân tích phân phối, mối quan hệ và insight
Scikit-learn	Huấn luyện và đánh giá mô hình	Cung cấp pipeline, preprocessing và nhiều thuật toán ML
Git, GitHub	Quản lý phiên bản	Hỗ trợ làm việc nhóm, branch, Pull Request và truy vết thay đổi
FastAPI	Xây dựng API dự đoán	Tách mô hình thành dịch vụ inference độc lập
Streamlit	Giao diện demo	Phù hợp xây dựng nhanh giao diện thử nghiệm bằng Python

1.6 Phương pháp và quy trình thực hiện

Dự án được tổ chức dựa trên tư tưởng của mô hình CRISP-DM (*Cross-Industry Standard Process for Data Mining*) là quy trình chuẩn trong ngành dùng để triển khai các dự án khai thác và phân tích dữ liệu. Trong đó quá trình phát triển đi từ việc hiểu bài toán nghiệp vụ, khảo sát dữ liệu, chuẩn bị dữ liệu, xây dựng mô hình, đánh giá đến triển khai. Các giai đoạn không hoàn toàn tuyến tính mà có thể quay lại kiểm tra và điều chỉnh khi phát hiện vấn đề trong dữ liệu hoặc kết quả mô hình.

Giai đoạn	Nội dung	Thành phần dự án
Business Understanding	Xác định bài toán, mục tiêu và tiêu chí đánh giá	NB00
Data Understanding	Khảo sát cấu trúc, chất lượng và insight dữ liệu	NB01, NB04
Data Preparation	Tổ chức CSDL, cleaning và feature engineering	NB02, NB03, NB05
Modeling	Huấn luyện và so sánh mô hình	NB06
Evaluation	Đánh giá và tối ưu ngưỡng dự đoán	NB06, NB06.1
Deployment	Đưa mô hình vào ứng dụng	FastAPI, Streamlit

Chương 2. Xây dựng pipeline và tiền xử lý dữ liệu

2.1. Quy trình tích hợp dữ liệu

2.1.1. Tổ chức dữ liệu raw trong PostgreSQL

Sau giai đoạn tìm hiểu dữ liệu, nhóm sử dụng **PostgreSQL** để tổ chức và quản lý dữ liệu phục vụ các bước xử lý tiếp theo. Database `credit_risk_db` được tạo để lưu trữ dữ liệu của dự án.

Dữ liệu ban đầu được tổ chức thành **7 bảng raw**:

- `applications`: hồ sơ vay hiện tại, chứa biến mục tiêu `TARGET`.
- `bureau`: lịch sử tín dụng của khách hàng tại các tổ chức tín dụng khác.
- `bureau_balance`: trạng thái tín dụng theo từng tháng của các khoản vay trong `bureau`.
- `previous_application`: lịch sử các hồ sơ vay trước đây tại Home Credit.
- `installments_payments`: lịch sử thanh toán theo từng kỳ.
- `pos_cash_balance`: lịch sử dư nợ và quá hạn của các khoản vay POS/Cash theo tháng.
- `credit_card_balance`: lịch sử dư nợ và thanh toán thẻ tín dụng theo tháng.

Dữ liệu từ các file CSV được import vào PostgreSQL bằng lệnh `COPY`. Danh sách cột được khai báo tường minh trong quá trình import nhằm bảo đảm dữ liệu được đưa vào đúng trường tương ứng.

Ba bảng có khóa định danh không trùng được khai báo **Primary Key**:

- `applications.sk_id_curr`
- `bureau.sk_id_bureau`

- `previous_application.sk_id_prev`

PostgreSQL tự động tạo unique index cho các Primary Key này. Các khóa nối không unique được bổ sung index ở bước tối ưu database nhằm hỗ trợ quá trình truy vấn và JOIN.

2.1.2. Xác định khóa và quan hệ giữa các bảng

Việc xác định đúng khóa liên kết giúp bảo đảm dữ liệu được tổng hợp và tích hợp đúng grain, tránh làm nhân bản hồ sơ khách hàng khi thực hiện JOIN.

Ba khóa định danh chính được sử dụng gồm:

- `SK_ID_CURR`: mã định danh hồ sơ vay hiện tại, được sử dụng để liên kết các bảng dữ liệu ở cấp khách hàng.
- `SK_ID_PREV`: mã định danh hồ sơ vay trước đây tại Home Credit.
- `SK_ID_BUREAU`: mã định danh khoản tín dụng của khách hàng tại tổ chức tín dụng khác.

Các quan hệ chính được sử dụng trong quá trình tổ chức dữ liệu:

- `applications` → `bureau` thông qua `SK_ID_CURR`.
- `bureau` → `bureau_balance` thông qua `SK_ID_BUREAU`.
- `applications` → `previous_application` thông qua `SK_ID_CURR`.
- `previous_application` → `installments_payments` thông qua `SK_ID_PREV`.
- `previous_application` → `pos_cash_balance` thông qua `SK_ID_PREV`.
- `previous_application` → `credit_card_balance` thông qua `SK_ID_PREV`.

Các bảng phụ có quan hệ **1-n**, do một khách hàng hoặc một hồ sơ vay trước có thể tương ứng với nhiều bản ghi lịch sử. Vì vậy, các bảng này không được nối trực tiếp ở grain chi tiết khi xây dựng dữ liệu cho mô hình mà cần được tổng hợp trước.

Quan hệ chi tiết giữa các bảng đã được trình bày trong sơ đồ ERD ở Chương 1.

2.1.3. Tổng hợp dữ liệu lịch sử

Các bảng phụ chứa nhiều bản ghi cho cùng một khách hàng hoặc cùng một khoản vay. Nếu nối trực tiếp các bảng này với applications, một hồ sơ khách hàng có thể bị nhân thành nhiều dòng, làm sai cấu trúc dữ liệu và ảnh hưởng đến quá trình xây dựng mô hình.

Vì vậy, nhóm thực hiện **Aggregation** trước khi tích hợp dữ liệu. Các phép tổng hợp chính gồm:

- COUNT: đếm số khoản vay, số hồ sơ hoặc số bản ghi lịch sử.
- SUM: tính tổng giá trị như tổng khoản vay, tổng dư nợ hoặc tổng số tiền thanh toán.
- AVG: tính giá trị trung bình của lịch sử khách hàng.
- MAX: xác định giá trị lớn nhất, đặc biệt hữu ích với các thông tin về mức độ quá hạn.
- MIN: xác định giá trị nhỏ nhất hoặc mốc lịch sử xa nhất.

Quá trình tổng hợp tạo ra **6 bảng summary**:

Bảng summary	Mức dữ liệu sau tổng hợp	Nội dung
bureau_balance_summary	1 dòng / SK_ID_BUREAU	Tổng hợp trạng thái theo tháng của từng khoản tín dụng
bureau_summary	1 dòng / SK_ID_CURR	Lịch sử tín dụng tại các tổ chức tín dụng khác
previous_application_summary	1 dòng / SK_ID_CURR	Lịch sử hồ sơ vay trước tại Home Credit
installments_payments_summary	1 dòng / SK_ID_CURR	Lịch sử thanh toán trả góp
pos_cash_balance_summary	1 dòng / SK_ID_CURR	Lịch sử POS/Cash
credit_card_balance_summary	1 dòng / SK_ID_CURR	Lịch sử sử dụng thẻ tín dụng

bureau_balance không chứa SK_ID_CURR mà chỉ có SK_ID_BUREAU. Vì vậy, bảng này được xử lý qua hai cấp:

- Đầu tiên, tổng hợp lịch sử theo SK_ID_BUREAU để tạo bureau_balance_summary.
- Sau đó, nối bureau_balance_summary với bureau thông qua SK_ID_BUREAU.

- Cuối cùng, dữ liệu được tổng hợp theo SK_ID_CURR để tạo bureau_summary.

Cách xử lý này đưa dữ liệu lịch sử theo tháng của từng khoản tín dụng về đúng cấp khách hàng trước khi tích hợp với applications.

	table_name text	total_rows bigint	distinct_keys bigint	result text
1	credit_card_balance_summary	103558	103558	OK
2	bureau_summary	305811	305811	OK
3	pos_cash_balance_summary	337252	337252	OK
4	installments_payments_summary	339587	339587	OK
5	previous_application_summary	338857	338857	OK
6	bureau_balance_summary	817395	817395	OK

Hình 2.1 Kết quả kiểm tra grain của các bảng summary

Kết quả kiểm tra cho thấy tổng số dòng bằng số khóa riêng biệt ở từng bảng summary. Điều này xác nhận bureau_balance_summary đạt mức một dòng trên mỗi SK_ID_BUREAU, trong khi năm bảng summary còn lại đạt mức một dòng trên mỗi SK_ID_CURR, đáp ứng điều kiện để thực hiện tích hợp dữ liệu mà không làm nhân bản khách hàng.

2.1.4. Tích hợp dữ liệu và tạo bảng application_flat

Sau khi các bảng lịch sử được tổng hợp về đúng grain, nhóm sử dụng applications làm bảng trung tâm để xây dựng bảng dữ liệu phẳng phục vụ các bước tiếp theo.

Năm bảng summary ở cấp SK_ID_CURR được tích hợp với applications gồm:

- bureau_summary
- previous_application_summary
- installments_payments_summary

- pos_cash_balance_summary
- credit_card_balance_summary

Riêng bureau_balance_summary không được nối trực tiếp với applications vì bảng này được tổng hợp ở cấp SK_ID_BUREAU và thông tin của nó đã được đưa vào bureau_summary.

Quá trình tích hợp sử dụng **LEFT JOIN theo SK_ID_CURR** nhằm:

- Giữ lại toàn bộ hồ sơ có trong applications.
- Bổ sung thông tin lịch sử từ các bảng summary khi khách hàng có dữ liệu tương ứng.
- Không loại bỏ khách hàng chỉ vì không có lịch sử ở một hoặc nhiều bảng phụ.
- Biểu diễn trường hợp không có lịch sử tương ứng bằng NULL để tiếp tục xem xét tại bước Data Cleaning.

Kết quả của quá trình tích hợp là bảng application_flat:

Chỉ tiêu	Kết quả
Số dòng	307.511
Số cột	180
Khóa chính	SK_ID_CURR
Grain	1 dòng / khách hàng
Biến mục tiêu	TARGET

Bảng application_flat gồm các thuộc tính gốc từ applications cùng các đặc trưng lịch sử đã được tổng hợp từ những bảng phụ.

2.1.5. Kiểm tra kết quả sau tích hợp

Sau khi tạo bảng application_flat, nhóm thực hiện các bước validation nhằm bảo đảm quá trình tổng hợp và tích hợp không làm thay đổi sai cấu trúc dữ liệu.

Các nội dung kiểm tra gồm:

- Kiểm tra tổng số dòng của application_flat.
- Kiểm tra số lượng SK_ID_CURR riêng biệt để phát hiện trường hợp nhân bản khách hàng.
- Kiểm tra tổng số cột sau khi tích hợp.
- Đối chiếu phân bố TARGET giữa applications và application_flat.
- Kiểm tra các giá trị NULL phát sinh từ các bảng summary sau LEFT JOIN.
- Kiểm tra kiểu dữ liệu của bảng sau khi tích hợp.

Chỉ tiêu kiểm tra	Mong đợi	Thực tế	Kết quả
Số dòng	307.511	307.511	Đạt
Số SK_ID_CURR riêng biệt	307.511	307.511	Đạt
Số cột	180	180	Đạt

2.2. Kịch bản kiểm thử dữ liệu sau khi Join

Sau khi tạo bảng application_flat, nhóm thực hiện một số kiểm tra để xác nhận quá trình tổng hợp và LEFT JOIN không làm mất hoặc nhân bản hồ sơ khách hàng. Kiểm tra đầu tiên nhằm xác nhận application_flat giữ đúng cấu trúc một khách hàng trên một dòng:

Kịch bản kiểm thử	Kết quả mong đợi	Kết quả thực tế	Đánh giá
Số dòng application_flat	307.511	307.511	Đạt
SK_ID_CURR duy nhất	307.511	307.511	Đạt
Số cột	180	180	Đạt
TARGET = 1	24.825	24.825	Đạt
TARGET = 0	282.686	282.686	Đạt
Grain bảng summary	Một dòng/ khóa	Đúng yêu cầu	Đạt
NULL sau LEFT JOIN	Khớp số khách không có lịch sử	Khớp kỳ vọng	Đạt

Nhận xét:

- Số lượng khách hàng và phân bố TARGET được bảo toàn sau khi Join.
- Các bảng summary đảm bảo đúng grain, không làm nhân bản khách hàng.

- Các giá trị NULL sau LEFT JOIN phù hợp với trường hợp khách hàng không có lịch sử tương ứng.
- application_flat đạt **307.511 dòng × 180 cột** và đủ điều kiện bàn giao cho bước Data Cleaning.

2.3. Làm sạch dữ liệu chuyên sâu

2.3.1. Đánh giá dữ liệu trước khi làm sạch

Dữ liệu đầu vào của bước làm sạch là bảng application_flat gồm **307.511 dòng × 180 cột**. Qua khảo sát ban đầu, nhóm ghi nhận:

- **125/180 cột** có giá trị thiếu.
- Một số nhóm lịch sử tín dụng có tỷ lệ thiếu cao do khách hàng không có lịch sử tương ứng.
- Một số giá trị bất thường được phát hiện như days_employed = 365243, code_gender = 'XNA' và bureau_sum_debt âm.
- Dữ liệu không phát sinh vấn đề đáng kể về bản ghi trùng lặp.
- Một số biến tiền tệ và dư nợ có phân phối lệch mạnh và xuất hiện giá trị cực đoan.

2.3.2. Phương pháp xử lý dữ liệu

Vấn đề	Phương pháp xử lý
Missing ở các nhóm lịch sử bureau, previous, installments, pos_cash, credit_card	Giữ NaN và tạo biến cờ xác định khách hàng có/không có lịch sử
Missing ở EXT_SOURCE và thông tin nhà ở	Điền bằng median
Missing ở biến phân loại	Điền nhãn "Missing"
Missing tỷ lệ rất thấp ở một số biến	Loại các dòng bị thiếu
days_employed = 365243	Tạo biến cờ, chuyển thành NaN và điền median
code_gender = 'XNA'	Loại bỏ do chỉ xuất hiện ở số lượng rất nhỏ
bureau_sum_debt < 0	Giới hạn về 0
Outlier ở các biến lệch mạnh	Capping tại phân vị P99

Việc xử lý được thực hiện theo ý nghĩa của từng nhóm dữ liệu thay vì áp dụng một phương pháp chung. Đặc biệt, các giá trị thiếu ở nhóm lịch sử tín dụng được giữ lại có chủ đích vì chúng có thể phản ánh khách hàng chưa có lịch sử tương ứng.

2.3.3. Xử lý Outlier

Dữ liệu tín dụng có nhiều biến tiền tệ, dư nợ và hạn mức mang phân phối lệch phải mạnh. Nhóm sử dụng **IQR để nhận diện outlier** và **skewness để đánh giá mức độ lệch** của từng biến. Các biến có độ lệch từ **3 trở lên** được lựa chọn để xử lý bằng phương pháp **capping tại phân vị 99 (P99)**.

Lý do lựa chọn P99:

- Các khoản vay, thu nhập hoặc dư nợ lớn vẫn có thể là giá trị hợp lệ, vì vậy **không nên xóa khách hàng chỉ vì giá trị cao**.
- Không sử dụng ngưỡng IQR để capping vì với dữ liệu lệch phải mạnh, ngưỡng IQR có thể quá thấp và làm thay đổi nhiều giá trị lớn nhưng hợp lệ.
- Không sử dụng Z-score vì **mean và độ lệch chuẩn dễ bị ảnh hưởng bởi chính các giá trị cực đoan**.
- P99 chỉ giới hạn phần cực trị phía trên của phân phối, giúp **giảm ảnh hưởng của các giá trị quá lớn nhưng vẫn giữ lại hồ sơ khách hàng**.
- Nhóm chỉ capping ngưỡng trên; các giá trị âm sai logic đã được xử lý riêng ở bước kiểm tra tính hợp lệ của dữ liệu.

Kết quả:

- Có **8 cột** có độ lệch từ 3 trở lên được áp dụng capping P99.
- Tổng cộng **22.446 giá trị** được giới hạn.
- Mức độ lệch của các biến cực đoan giảm đáng kể; ví dụ amt_income_total giảm skewness từ **392,04 xuống 1,31**.
- Không có khách hàng nào bị loại bỏ do xử lý outlier.

2.3.4. Kết quả sau làm sạch

Chỉ số	Trước làm sạch	Sau làm sạch
Số dòng	307.511	306.195
Số cột	180	188
Tổng số ô missing	12.456.340	3.280.224
Số cột có missing	125	58
Giá trị sai logic đã xác định	56.674	0
Dòng trùng lặp	0	0
Giá trị được cap P99	0	22.446

Nhận xét:

- Dữ liệu giảm từ **307.511 xuống 306.195 hồ sơ**, chủ yếu do loại các dòng thiếu ở một số thuộc tính có tỷ lệ missing thấp và các giá trị code_gender = 'XNA'.
- Số cột tăng từ **180 lên 188** do bổ sung các biến cờ trong quá trình làm sạch.
- Tổng số ô missing giảm đáng kể; phần missing còn lại chủ yếu thuộc các nhóm lịch sử và được giữ có chủ đích.
- Các trường hợp sai logic đã xác định được xử lý hoàn toàn.

- Bảng application_flat_cleaned cuối cùng có **306.195 dòng × 188 cột**, sẵn sàng cho EDA và Feature Engineering.

Chương 3. Phân tích sâu dữ liệu và tìm ý tưởng tạo feature

3.1. Phân tích khám phá dữ liệu (EDA)

3.1.1. Chuẩn bị dữ liệu và phương pháp EDA

Nhóm sử dụng bảng application_flat_cleaned sau bước làm sạch, gồm **306.195 hồ sơ khách hàng**, để thực hiện phân tích khám phá dữ liệu. Tỷ lệ TARGET = 1 trên tập dữ liệu là khoảng **8,09%** và được sử dụng làm **mức rủi ro nền** để so sánh giữa các phân khúc khách hàng trong toàn bộ quá trình EDA.

Các biến được tổ chức thành **9 nhóm nghiệp vụ** gồm: nhân khẩu học, gia đình và công việc; tài chính và khoản vay; lịch sử tín dụng; nhà ở và tài sản; hồ sơ đăng ký vay; khu vực và quan hệ xã hội; liên lạc và giấy tờ; điểm đánh giá bên ngoài; và lịch sử tra cứu tín dụng.

Với từng nhóm, nhóm thống kê **số lượng khách hàng** và **tỷ lệ TARGET = 1**, sau đó sử dụng các hình thức trực quan phù hợp như biểu đồ cột, histogram, scatter plot và heatmap. Việc xem xét đồng thời tỷ lệ rủi ro và quy mô mẫu giúp hạn chế các kết luận sai lệch từ những phân khúc có quá ít quan sát.

3.1.2. Phân tích đơn biến theo các nhóm nghiệp vụ

Phân tích đơn biến được thực hiện trên 9 nhóm biến nghiệp vụ nhằm xác định những đặc điểm có khả năng phân tách rủi ro tốt và loại bỏ các biến có tín hiệu yếu hoặc trùng lặp. Kết quả cho thấy nhiều biến không có quan hệ tuyến tính rõ với TARGET, vì vậy nhóm không chỉ xem xét giá trị gốc mà còn đánh giá các biến tỷ lệ, cờ trạng thái và các nhóm phân khúc.

Nhóm biến	Kết quả nổi bật từ EDA	Định hướng
Nhân khẩu học, gia đình và công việc	Tỷ lệ rủi ro giảm rõ theo tuổi, từ 12,41% ở nhóm dưới 25 tuổi xuống 5,22% ở nhóm từ 55 tuổi. Thâm niên làm việc, học vấn và nghề nghiệp cũng tạo khác biệt đáng kể.	Giữ tuổi, thâm niên, học vấn, nghề nghiệp và một số biến gia đình để tiếp tục đánh giá.
Tài chính và khoản vay	Thu nhập, khoản vay và tiền trả góp riêng lẻ có quan hệ phi tuyến. LTV > 120% có tỷ lệ rủi ro 11,72% , cao hơn mức nền 8,09%.	Ưu tiên các tỷ số như ltv, dti_proxy, credit_to_income.
Lịch sử tín dụng	Dư nợ và hành vi chậm trả cho tín hiệu mạnh hơn số lượng khoản vay. Nhóm không còn dư nợ có tỷ lệ khoảng 5,58% , trong khi tỷ lệ dư nợ cao đi kèm rủi ro lớn hơn.	Ưu tiên tỷ lệ dư nợ, tỷ lệ chậm trả, trạng thái quá hạn và độ gần đây của lịch sử tín dụng.

Nhà ở và tài sản	Thuê nhà và ở cùng bố mẹ có tỷ lệ rủi ro cao hơn; nhiều chỉ số định lượng về diện tích và kết cấu nhà có tương quan rất yếu với TARGET.	Giữ loại hình nhà ở và một số cờ tài sản; không ưu tiên nhóm biến định lượng nhà ở.
Hồ sơ đăng ký vay	Giờ nộp hồ sơ, độ mới của giấy tờ và thời gian đăng ký cư trú có tín hiệu; thứ trong tuần gần như không tạo khác biệt.	Giữ các biến thời gian có tín hiệu, không ưu tiên weekday_appr_process_start.
Khu vực và quan hệ xã hội	Xếp hạng khu vực và các cờ khác thành phố phân tách rủi ro rõ. Quan hệ xã hội có dấu hiệu quá hạn mạnh hơn số lượng người được quan sát.	Ưu tiên một biến xếp hạng khu vực đại diện và các cờ quá hạn xã hội.
Liên lạc và giấy tờ	flag_mobil là cột hằng; một số cờ liên lạc và giấy tờ có tín hiệu nhưng dễ bị trùng lặp hoặc ảnh hưởng bởi nhóm hiếm.	Loại cột hằng, giữ có chọn lọc các cờ đủ quy mô và có khả năng diễn giải.

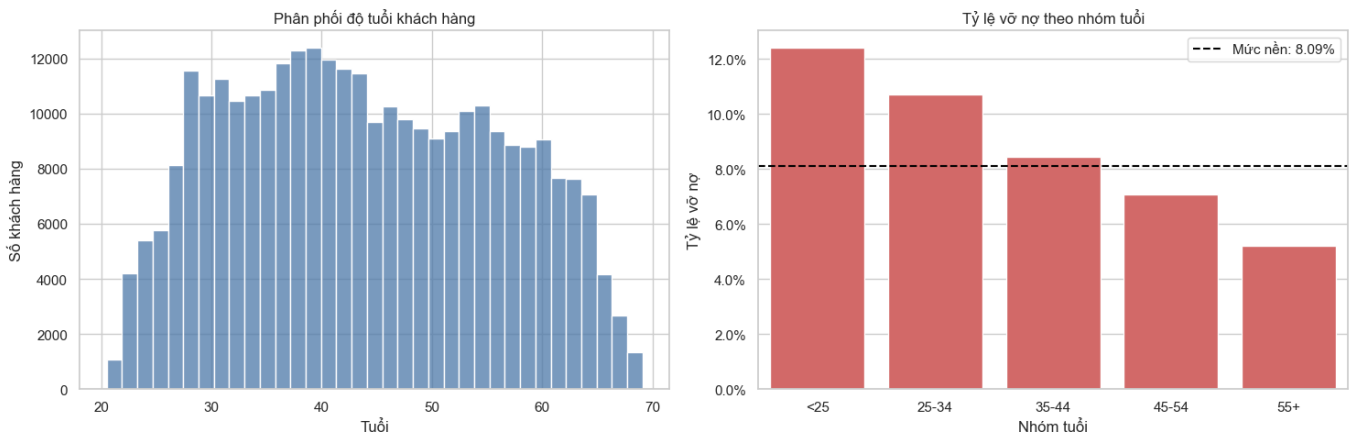
Điểm đánh giá bên ngoài	EXT_SOURCE_1, EXT_SOURCE_2, EXT_SOURCE_3 đều phân tách rủi ro rõ. Điểm tổng hợp ext_sources_mean giảm từ 28,64% ở nhóm điểm thấp xuống 1,29% ở nhóm điểm cao.	Đây là nhóm biến được ưu tiên cao khi xây dựng feature.
Lịch sử tra cứu tín dụng	Nhóm thiếu thông tin tra cứu có tỷ lệ rủi ro 10,36% , cao hơn nhóm có thông tin 7,74% . Biến tra cứu theo năm ổn định hơn các cửa sổ ngắn.	Giữ has_credit_bureau_request_info và ưu tiên amt_req_credit_bureau_year.

Từ kết quả trên, có thể rút ra ba đặc điểm chính của dữ liệu:

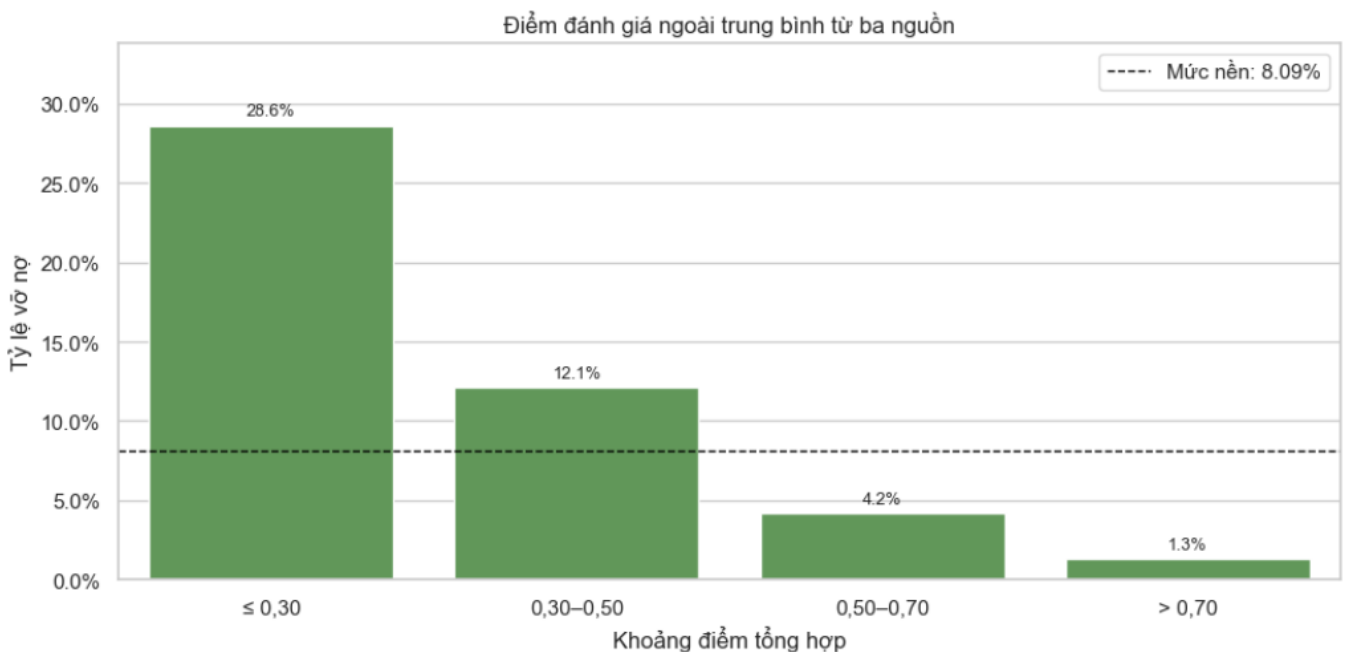
Thứ nhất, các biến tỷ lệ thường cung cấp tín hiệu rõ hơn các giá trị tuyệt đối, điển hình là tỷ lệ dư nợ, tỷ lệ chậm trả và LTV.

Thứ hai, nhiều quan hệ với rủi ro mang tính **phi tuyến**, nên không thể đánh giá chỉ dựa trên tương quan tuyến tính.

Thứ ba, một số trạng thái thiếu dữ liệu hoặc cờ đặc biệt vẫn chứa thông tin có giá trị và cần được giữ lại thay vì xử lý đồng nhất.



Hình 3.1 Biểu đồ phân phối độ tuổi khách hàng và Tỷ lệ vỡ nợ theo nhóm tuổi



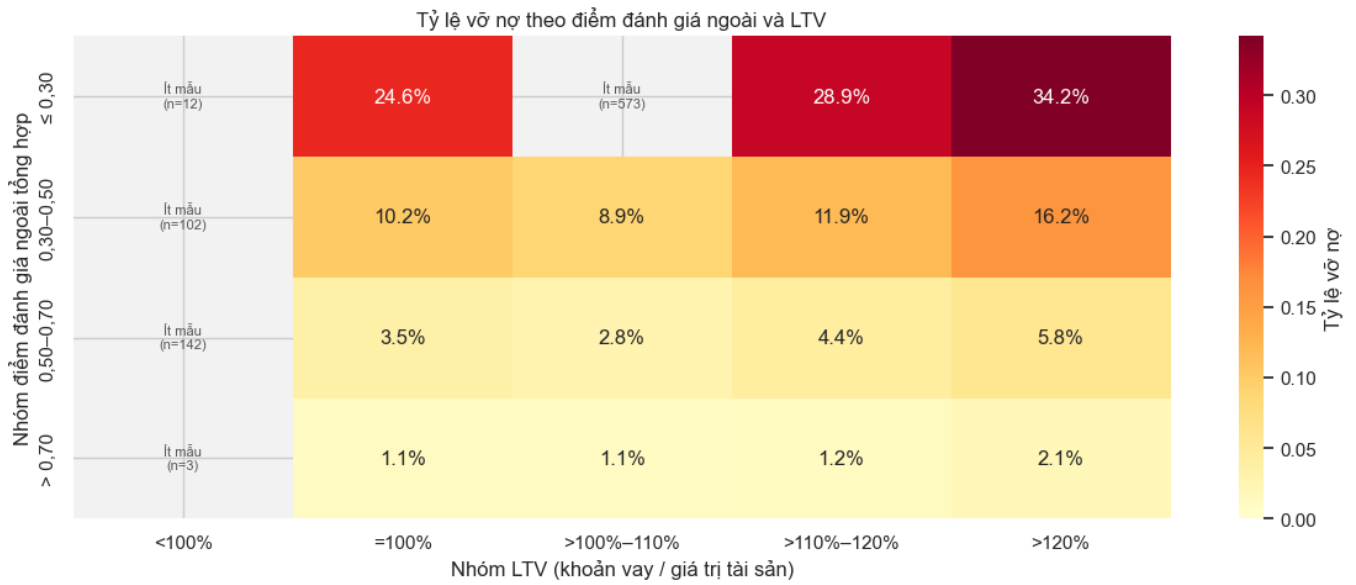
Hình 3.2 Biểu đồ Điểm đánh giá ngoài

3.1.3. Phân tích đa biến và tương tác

Sau phân tích đơn biến, nhóm tiếp tục thực hiện EDA đa biến nhằm kiểm tra xem tác động của một yếu tố rủi ro có thay đổi khi kết hợp với một yếu tố khác hay không. Để hạn chế kết luận từ các phân khúc có quy mô quá nhỏ, các tổ hợp dưới **1.000 khách hàng** được đánh dấu là ít mẫu và không được sử dụng để đưa ra kết luận chính.

Phân tích tương tác	Kết quả chính	Ý nghĩa
Tuổi × Thu nhập	Trong cùng mức thu nhập, khách hàng trẻ vẫn có tỷ lệ rủi ro cao hơn rõ rệt. Nhóm dưới 25 tuổi đạt khoảng 10,75%–13,34% , trong khi nhóm 55+ chỉ khoảng 4,63%–5,41% ở các tổ hợp đủ mẫu.	Tuổi vẫn là tín hiệu rủi ro đáng chú ý ngay cả khi đã xem xét cùng thu nhập.
Lịch sử chậm trả × Dư nợ	Ở mọi mức dư nợ, nhóm từng có dấu hiệu chậm trả đều có tỷ lệ rủi ro cao hơn. Nhóm từng trễ và còn dư nợ cao/rất cao đạt khoảng 10,36%–10,54% .	Lịch sử thanh toán và gánh nặng dư nợ có tác động kết hợp, phù hợp để tạo biến tương tác.
Điểm đánh giá ngoài × LTV	Nhóm có điểm ngoài tổng hợp $\leq 0,30$ và LTV > 120% đạt tỷ lệ rủi ro 34,19% , cao nhất trong các tổ hợp đa biến được phân tích.	Điểm tín dụng thấp kết hợp mức vay cao so với giá trị tài sản tạo thành phân khúc rủi ro nổi bật.

Kết quả đa biến cho thấy rủi ro tín dụng không chỉ phụ thuộc vào từng biến riêng lẻ. Một số yếu tố có thể **khuyến đại rủi ro khi xuất hiện đồng thời**, đặc biệt là điểm đánh giá ngoài thấp kết hợp với LTV cao. Đây là cơ sở để NB05 tiếp tục thử nghiệm các biến tương tác thay vì chỉ sử dụng các đặc trưng gốc độc lập.



Hình 3.3 Biểu đồ heatmap Tỷ lệ vỡ nợ theo điểm đánh giá ngoài và LTV

3.1.4. Kết luận EDA và định hướng Feature Engineering

Kết quả EDA cho thấy rủi ro tín dụng không được giải thích tốt bởi một biến đơn lẻ mà chịu ảnh hưởng đồng thời từ đặc điểm khách hàng, khả năng tài chính, lịch sử tín dụng và các tín hiệu đánh giá bên ngoài. Nhiều biến có quan hệ phi tuyến với TARGET, trong khi các biến tỷ lệ và biến tương tác thường thể hiện khả năng phân tách rủi ro tốt hơn các giá trị tuyệt đối.

Kết luận từ EDA	Định hướng Feature Engineering
Các giá trị tài chính tuyệt đối có quan hệ phi tuyến	Tạo các tỷ số như LTV, DTI, Credit/Income
Dư nợ và tần suất chậm trả có tín hiệu mạnh	Tạo tỷ lệ dư nợ, tỷ lệ chậm trả và các cờ quá hạn
Missing ở một số nhóm mang thông tin	Giữ hoặc tạo cờ trạng thái thiếu/ lịch sử thay vì xử lý đồng nhất
Một số biến có thông tin gần trùng nhau	Lựa chọn biến đại diện hoặc kiểm tra mức đóng góp trước khi giữ đồng thời
EXT_SOURCE có khả năng phân tách rủi ro mạnh	Giữ ba điểm gốc và thử điểm tổng hợp
Các heatmap cho thấy hiệu ứng kết hợp giữa các yếu tố	Thử các biến tương tác , đặc biệt điểm ngoài × LTV và chậm trả × dư nợ

Từ các kết quả trên, nhóm xây dựng danh sách đặc trưng tiềm năng để chuyển sang bước Feature Engineering. Các feature này không chỉ nhằm tăng khả năng dự báo mà còn ưu tiên tính diễn giải theo nghiệp vụ, khả năng tái tạo từ dữ liệu đầu vào và hạn chế các biến có tín hiệu yếu hoặc trùng lặp.

3.2. Xây dựng biến phái sinh (Feature Engineering)

Dựa trên kết quả EDA, nhóm xây dựng feature từ bảng `application_flat_cleaned` gồm 306.195 dòng $\times$ 188 cột. Các bảng lịch sử đã được tổng hợp về mức một khách hàng tại NB02, vì vậy NB05 không đọc lại dữ liệu raw và không làm phát sinh thêm dòng. Mục tiêu là chuyển thông tin gốc thành các chỉ số dễ diễn giải hơn về khả năng chi trả, mức độ ổn định, lịch sử tín dụng và tương tác giữa các yếu tố rủi ro.

Quy mô feature: NB05 tạo 46 feature mới theo bảy nhóm nguồn/nghiệp vụ, nâng dữ liệu từ 188 lên 234 cột mà vẫn giữ nguyên 306.195 khách hàng, `SK_ID_CURR` và `TARGET`.

Nguồn dữ liệu	Feature cần tạo	Cách tính	Ý nghĩa
applications	ltv	$\text{amt_credit} / \text{amt_goods_price}$	Tỷ lệ khoản vay so với giá trị tài sản mua.
applications	credit_term	$\text{amt_credit} / \text{amt_annuity}$	Số kỳ trả tương đối của khoản vay.
applications	credit_to_income	$\text{amt_credit} / \text{amt_income_total}$	Quy mô khoản vay so với thu nhập.
applications	dti	$\text{amt_annuity} / \text{amt_income_total}$	Tỷ lệ khoản trả góp hiện tại so với thu nhập.
applications	income_per_person	$\text{amt_income_total} / \text{cnt_fam_members}$	Thu nhập bình quân theo thành viên gia đình.
applications	age_years	$-\text{days_birth} / 365.25$	Tuổi của khách hàng tại thời điểm nộp hồ sơ.

applications	employment_years	-days_employed / 365.25	Số năm làm việc hiện tại của khách hàng.
applications	employment_age_ratio	days_employed / days_birth	Thời gian làm việc tương đối so với tuổi đời.
applications	id_publish_years	-days_id_publish / 365.25	Số năm kể từ lần cấp giấy tờ gần nhất.
applications	application_hour_group	Nhóm hour_appr_process_start theo khung giờ	Khung giờ khách hàng nộp hồ sơ.
applications	ext_sources_mean	Trung bình ba ext_source_*	Mức đánh giá trung bình từ các nguồn bên ngoài.
applications	ext_sources_min	Giá trị nhỏ nhất của ba ext_source_*	Giữ lại điểm đánh giá thấp nhất.
applications	ext_sources_std	Độ lệch chuẩn của ba ext_source_*	Mức chênh lệch giữa các nguồn đánh giá.

applications	registration_years	-days_registration / 365.25	Số năm kể từ khi khách hàng thay đổi đăng ký gần nhất.
applications	phone_change_years	-days_last_phone_change / 365.25	Số năm kể từ lần thay đổi số điện thoại gần nhất.
applications	address_mismatch_count	Tổng 6 cờ không khớp nơi đăng ký, nơi ở và nơi làm việc	Số dấu hiệu địa chỉ cư trú hoặc làm việc không đồng nhất.
applications	social_default_ratio_30	def_30_cnt_social_circle / obs_30_cnt_social_circle	Tỷ lệ người trong vòng quan hệ xã hội có dấu hiệu vỡ nợ trong 30 ngày.
bureau	bureau_debt_ratio	bureau_sum_debt / bureau_sum_credit	Tỷ lệ dư nợ còn lại tại tổ chức khác.
bureau	has_bureau_overdue	bureau_max_overdue > 0	Cờ từng quá hạn tại tổ chức khác.

bureau	bureau_recency_days	-bureau_latest_days_credit	Số ngày từ khoản tín dụng Bureau gần nhất.
bureau	bureau_active_ratio	bureau_active_count / bureau_count	Tỷ lệ khoản tín dụng Bureau đang hoạt động.
bureau	bureau_recent_12m_ratio	bureau_recent_loan_12m_count / bureau_count	Tỷ lệ khoản tín dụng Bureau phát sinh trong 12 tháng gần nhất.
bureau	bureau_overdue_loan_ratio	bureau_overdue_loan_count / bureau_count	Tỷ lệ khoản tín dụng Bureau từng quá hạn.
bureau	bureau_debt_to_income	bureau_sum_debt / amt_income_total	Dư nợ Bureau so với thu nhập hiện tại của khách hàng.
bureau_balance	bureau_balance_dpd_month_ratio	bureau_balance_dpd_month_count / bureau_balance_month_count	Tỷ lệ số tháng ghi nhận trạng thái quá hạn trong lịch sử Bureau.

previous_application	previous_credit_t o_current	previous_avg_credit / amt_credit	Quy mô khoản vay cũ trung bình so với khoản vay hiện tại.
previous_application	previous_recency _days	-previous_latest_decision	Số ngày từ hồ sơ vay trước gần nhất.
previous_application	previous_approva l_rate	previous_approved_count / previous_count	Tỷ lệ hồ sơ vay trước đã được duyet.
previous_application	previous_refusal_ rate	previous_refused_count / previous_count	Tỷ lệ hồ sơ vay trước bị từ chối.
previous_application	previous_recent_ 12m_ratio	previous_recent_12m_count / previous_count	Tỷ lệ hồ sơ vay trước phát sinh trong 12 tháng gần nhất.
installments_pay ments	installments_pay ment_ratio	installments_sum_paid / installments_sum_due	Tỷ lệ số tiền đã trả so với số tiền phải trả.

installments_payments	has_installments_late	installments_max_late > 0	Cờ từng trả góp trễ hạn.
installments_payments	installments_late_rate	installments_late_count / installments_count	Tỷ lệ kỳ trả góp từng thanh toán trễ hạn.
installments_payments	installments_recent_12m_late_rate	installments_recent_12m_late_count / installments_recent_12m_count	Tỷ lệ kỳ trả trễ trong 12 tháng gần nhất.
installments_payments	installments_recent_12m_underpaid_rate	installments_recent_12m_underpaid_count / installments_recent_12m_count	Tỷ lệ kỳ trả thiếu trong 12 tháng gần nhất.
credit_card_balance	credit_card_utilization	credit_card_avg_balance / credit_card_avg_limit	Mức sử dụng hạn mức thẻ trung bình.
credit_card_balance	credit_card_balance_to_income	credit_card_avg_balance / amt_income_total	Dư nợ thẻ tín dụng trung bình so với thu nhập hiện tại.

pos_cash_balance	has_pos_cash_dpd	$\text{pos_cash_max_dpd} > 0$	Cờ từng quá hạn khoản vay POS/Cash.
pos_cash_balance	pos_cash_history_months	$\text{pos_cash_latest_month} - \text{pos_cash_oldest_month} + 1$	Độ dài lịch sử POS/Cash được ghi nhận theo tháng.
applications	age_income_interaction	Nhóm tuổi $\times$ nhóm thu nhập	Kết hợp tuổi với mức thu nhập.
Nhiều nguồn lịch sử	late_debt_interaction	Lịch sử trễ hạn $\times$ nhóm dư nợ	Kết hợp hành vi trễ hạn với mức dư nợ.
applications	ext_ltv_interaction	$(1 - \text{ext_sources_mean}) \times \text{ltv}$	Kết hợp điểm ngoài thấp với LTV cao.
applications	ext_credit_income_interaction	$(1 - \text{ext_sources_mean}) \times \text{credit_to_income}$	Kết hợp điểm ngoài thấp với khoản vay cao so với thu nhập.

applications	ext_dti_interaction	$(1 - \text{ext_sources_mean}) \times \text{dti}$	Kết hợp điểm ngoài thấp với áp lực trả góp cao.
applications	ext_min_ltv_interaction	$(1 - \text{ext_sources_min}) \times \text{ltv}$	Kết hợp điểm ngoài thấp nhất với LTV cao.
applications và bureau	ext_bureau_debt_interaction	$(1 - \text{ext_sources_mean}) \times \text{bureau_debt_ratio}$	Kết hợp điểm ngoài thấp với tỷ lệ dư nợ Bureau cao.

Ghi chú: Với các feature dạng tỷ lệ, mẫu số bằng 0 được chuyển thành NaN trước khi thực hiện phép chia để tránh phát sinh giá trị vô cực (inf). Các giá trị thiếu có ý nghĩa nghiệp vụ được giữ lại và tiếp tục xử lý trong pipeline của Notebook 06.

Hai feature age_income_interaction và late_debt_interaction sử dụng các ngưỡng phân vị tính trên toàn bộ dữ liệu trong Notebook 05 nên **không được sử dụng trực tiếp khi huấn luyện mô hình**. Notebook 06 phải tính lại các ngưỡng phân vị từ tập train và áp dụng các ngưỡng đó cho validation/test nhằm tránh data leakage.

Tổng cộng Notebook 05 tạo **46 feature mới**, trong đó **44 feature được bàn giao trực tiếp** sang Notebook 06 và **2 feature tương tác sử dụng phân vị phải được tạo lại từ tập train**.

Chương 4. Mô hình hóa và dự báo

4.1. Xây dựng mô hình dự báo rủi ro tín dụng

Bài toán của dự án là dự báo khả năng khách hàng gặp khó khăn trong thanh toán khoản vay, với biến mục tiêu target chỉ nhận hai giá trị 0 và 1. Vì vậy, nhóm xác định đây là bài toán **phân loại nhị phân (Binary Classification)**. Mục tiêu của mô hình là ước lượng xác suất một khách hàng thuộc nhóm có nguy cơ nợ xấu, từ đó hỗ trợ quá trình sàng lọc và thẩm định hồ sơ vay.

Dữ liệu đầu vào được lấy từ bảng application_features trong PostgreSQL, gồm **306.195 dòng × 234 cột**. Sau khi loại hai cột sk_id_curr và target, dữ liệu được chia có phân tầng (stratify) thành ba tập theo tỷ lệ xấp xỉ **64% Train – 16% Validation – 20% Test**. Tập Train được sử dụng để học các bước tiền xử lý và huấn luyện mô hình; tập Validation được sử dụng để đánh giá, so sánh và lựa chọn mô hình; tập Test được giữ riêng và không tham gia vào quá trình lựa chọn mô hình tại Notebook 06.

Hai interaction feature age_income_interaction và late_debt_interaction phụ thuộc vào các ngưỡng phân vị của dữ liệu. Vì phiên bản được tạo ở giai đoạn Feature Engineering sử dụng phân vị của toàn bộ dữ liệu, hai feature này được loại khỏi tập đầu vào trước khi chia dữ liệu và được tạo lại sau đó. Các ngưỡng thu nhập và dư nợ chỉ được học từ tập Train rồi áp dụng cố định cho Validation và Test, nhằm hạn chế Data Leakage.

Các bước tiền xử lý khác cũng được xác định hoặc fit từ tập Train. Nhóm loại các cột hằng số, xử lý giá trị thiếu theo ý nghĩa nghiệp vụ, sử dụng median của Train cho các biến số còn thiếu, gán nhãn Unknown cho các biến phân loại thiếu và thực hiện One-Hot Encoding cho các biến phân loại. Logistic Regression sử dụng thêm dữ liệu số đã được chuẩn hóa bằng StandardScaler, trong khi Random Forest và HistGradientBoosting sử dụng dữ liệu đã tiền xử lý nhưng không cần chuẩn hóa.

Sau quá trình tiền xử lý, Train, Validation và Test có cùng cấu trúc feature, không còn giá trị thiếu và không chứa giá trị vô cực. Việc toàn bộ tham số tiền xử lý chỉ được học từ Train rồi áp dụng sang Validation và Test giúp hạn chế rò rỉ thông tin và bảo đảm các mô hình được đánh giá trong cùng một điều kiện.

Nhóm xây dựng và so sánh ba thuật toán phân loại:

- **Logistic Regression:** được sử dụng làm mô hình baseline vì cấu trúc đơn giản, tốc độ huấn luyện nhanh và có khả năng diễn giải thông qua hệ số của từng feature.
- **Random Forest:** kết hợp nhiều cây quyết định để học các quan hệ phi tuyến và tương tác phức tạp giữa đặc điểm khách hàng và rủi ro tín dụng.
- **HistGradientBoosting:** xây dựng các cây theo cơ chế boosting, trong đó các cây sau tập trung cải thiện những trường hợp mà các cây trước dự đoán chưa tốt. Thuật toán sử dụng histogram để tăng hiệu quả huấn luyện trên tập dữ liệu lớn.

Cả ba mô hình được huấn luyện trên cùng tập Train và đánh giá trên cùng tập Validation tại ngưỡng tạm thời 0,5. Kết quả cho thấy HistGradientBoosting đạt hiệu quả tổng thể tốt nhất về khả năng phân biệt và xếp hạng rủi ro. Vì vậy, nhóm lựa chọn **HistGradientBoosting** làm mô hình ứng viên để tiếp tục tối ưu ngưỡng dự đoán tại Notebook 06.1. Tập Test vẫn được giữ riêng và chỉ được sử dụng ở bước đánh giá cuối cùng sau khi mô hình và ngưỡng dự đoán đã được cố định.

4.2. Đánh giá hiệu suất mô hình

Dữ liệu của dự án có sự mất cân bằng rõ rệt giữa hai lớp, trong đó tỷ lệ khách hàng thuộc nhóm nợ xấu ($\text{target} = 1$) chỉ khoảng **8,1%**. Vì vậy, nhóm không sử dụng Accuracy làm tiêu chí chính để lựa chọn mô hình. Nếu chỉ dựa vào Accuracy, mô hình có thể đạt kết quả cao bằng cách dự đoán phần lớn khách hàng thuộc nhóm không nợ xấu, nhưng vẫn bỏ sót nhiều khách hàng thực sự có rủi ro.

Do đó, nhóm sử dụng đồng thời các chỉ số ROC-AUC, PR-AUC, Precision, Recall và F1-score để đánh giá hiệu suất mô hình:

- **ROC-AUC:** phản ánh khả năng mô hình xếp khách hàng nợ xấu có mức rủi ro cao hơn khách hàng trả được nợ trên nhiều ngưỡng dự đoán khác nhau.
- **PR-AUC:** phản ánh mối quan hệ giữa Precision và Recall, đặc biệt phù hợp khi lớp dương chiếm tỷ lệ thấp như trong bài toán tín dụng của dự án.
- **Precision:** cho biết trong số khách hàng bị mô hình cảnh báo là nợ xấu, có bao nhiêu khách hàng thực sự thuộc nhóm nợ xấu.
- **Recall:** cho biết trong số khách hàng thực sự nợ xấu, mô hình phát hiện được bao nhiêu trường hợp.
- **F1-score:** là trung bình điều hòa giữa Precision và Recall, giúp đánh giá sự cân bằng giữa khả năng phát hiện khách hàng rủi ro và số lượng cảnh báo nhầm.

RMSE không được sử dụng vì đây là bài toán phân loại, không phải bài toán hồi quy.

Để bảo đảm việc so sánh công bằng, cả ba mô hình được huấn luyện trên cùng tập Train và đánh giá trên cùng tập Validation tại ngưỡng tạm thời 0,5. Tập Test chưa được sử dụng trong giai đoạn này và được giữ riêng cho bước đánh giá cuối cùng sau khi mô hình và ngưỡng dự đoán đã được cố định.

Kết quả đánh giá trên tập Validation được trình bày trong bảng sau:

Mô hình	ROC-AUC	PR-AUC	Precision	Recall	F1-score
Logistic Regression	0,7667	0,2463	0,1729	0,6881	0,2763
Random Forest	0,7601	0,2438	0,1877	0,5985	0,2858
HistGradientBoosting	0,7745	0,2598	0,1829	0,6765	0,2879

HistGradientBoosting đạt kết quả tốt nhất về **ROC-AUC 0,7745**, **PR-AUC 0,2598** và **F1-score 0,2879**. Điều này cho thấy mô hình có khả năng phân biệt và xếp hạng rủi ro tốt hơn hai mô hình còn lại trên tập Validation.

Logistic Regression đạt Recall cao nhất với 0,6881, nghĩa là phát hiện được tỷ lệ khách hàng nợ xấu cao hơn. Tuy nhiên, Precision chỉ đạt 0,1729, thấp nhất trong ba mô hình, cho thấy số lượng khách hàng tốt bị cảnh báo nhầm tương đối lớn.

Random Forest đạt Precision cao nhất với 0,1877, nhưng Recall chỉ đạt 0,5985, thấp nhất trong ba mô hình. Điều này cho thấy Random Forest đưa ra các cảnh báo có độ chính xác cao hơn một chút, nhưng đồng thời bỏ sót nhiều khách hàng nợ xấu hơn. PR-AUC của Random Forest cũng chỉ đạt 0,2438, thấp hơn HistGradientBoosting.

HistGradientBoosting duy trì Recall tương đối cao ở mức 0,6765, đồng thời đạt ROC-AUC, PR-AUC và F1-score cao nhất. Vì vậy, xét tổng thể khả năng phân biệt rủi ro và sự cân bằng giữa Precision–Recall, nhóm lựa chọn **HistGradientBoosting** làm mô hình ứng viên cuối cùng.

Tuy nhiên, ngưỡng 0,5 ở giai đoạn này chỉ được sử dụng để so sánh các mô hình trong cùng điều kiện, chưa được xem là ngưỡng dự đoán cuối cùng. Sau khi HistGradientBoosting được

lựa chọn, quá trình tối ưu ngưỡng được thực hiện riêng tại Notebook 06.1 trên tập Validation theo **F2-score**, với mục tiêu ưu tiên Recall và hạn chế các trường hợp False Negative.

4.3. Quá trình kiểm chứng và cải tiến feature sau đánh giá mô hình

Sau khi hoàn thành giai đoạn Feature Engineering, nhóm không xem toàn bộ feature được tạo ra là có giá trị như nhau. Các feature mới tiếp tục được kiểm chứng thông qua kết quả mô hình và Permutation Importance để đánh giá mức độ đóng góp thực tế vào khả năng dự báo rủi ro tín dụng.

Quá trình kiểm chứng được thực hiện theo ba vòng chính. Mỗi vòng tập trung vào một nhóm giả thuyết feature khác nhau, sau đó đánh giá lại trong cùng pipeline mô hình hóa để hạn chế việc giữ feature chỉ vì có ý nghĩa nghiệp vụ nhưng không tạo thêm tín hiệu dự báo.

Vòng	Trọng tâm kiểm chứng	Kết quả chính	Quyết định
Vòng 1	Bổ sung các summary feature liên quan đến số lượng hợp đồng, mức trả thiếu, trễ hạn gần đây và hành vi sử dụng thẻ tín dụng	Một số feature cung cấp thêm tín hiệu nhưng mức đóng góp giữa các biến không đồng đều	Giữ lại các feature có ý nghĩa và tiếp tục kiểm chứng trong mô hình tổng thể
Vòng 2	Kiểm chứng giả thuyết rằng hành vi tín dụng trong 12 tháng gần nhất phản ánh rủi ro tốt hơn lịch sử dài hạn	Một số feature gần thời điểm hiện tại cung cấp tín hiệu hữu ích hơn trong quá trình phân loại rủi ro	Tiếp tục giữ các feature có tín hiệu rõ để đánh giá cùng toàn bộ tập feature
Vòng 3	Bổ sung các interaction giữa điểm đánh giá ngoài và các chỉ số thể hiện áp lực tài chính	Các interaction mới có ý nghĩa nghiệp vụ nhưng mức đóng góp bổ sung không đồng đều và có thể trùng lặp một phần với các feature nguồn	Không tiếp tục mở rộng interaction chỉ dựa trên giả thuyết nghiệp vụ

Đối với HistGradientBoosting, nhóm sử dụng **Permutation Importance** để đánh giá mức độ mô hình phụ thuộc vào từng feature. Phương pháp này lần lượt hoán đổi ngẫu nhiên giá trị của một feature rồi đo mức suy giảm hiệu suất của mô hình. Trong dự án, **PR-AUC** được sử dụng làm scoring metric cho Permutation Importance vì dữ liệu có sự mất cân bằng giữa hai lớp và lớp nợ xấu chiếm tỷ lệ thấp.

Permutation Importance được tính trên một mẫu cố định lấy từ **tập Validation**, không sử dụng tập Test. Nhờ đó, tập Test vẫn được giữ độc lập và chỉ được sử dụng ở giai đoạn đánh giá cuối cùng.

Kết quả kiểm chứng cho thấy một số feature có mức đóng góp rõ ràng hơn các feature khác, trong khi nhiều feature mới chỉ bổ sung tín hiệu nhỏ. Đặc biệt, với các interaction feature, việc có ý nghĩa hợp lý về mặt nghiệp vụ không đồng nghĩa với việc chúng chắc chắn cải thiện hiệu suất mô hình. Một phần thông tin của interaction có thể đã được phản ánh thông qua các feature nguồn, khiến mức đóng góp bổ sung không lớn.

Nhóm cũng không xem Feature Importance hoặc Permutation Importance là bằng chứng về quan hệ nhân quả. Giá trị importance chỉ phản ánh mức độ mô hình hiện tại phụ thuộc vào feature trong tập dữ liệu đang được đánh giá. Vì vậy, một feature có importance cao không đồng nghĩa với việc feature đó trực tiếp gây ra nợ xấu.

Qua ba vòng kiểm chứng, nhóm rút ra rằng quá trình xây dựng feature cần kết hợp giữa **hiểu biết nghiệp vụ và đánh giá thực nghiệm**. Feature mới chỉ nên được ưu tiên khi vừa có ý nghĩa hợp lý về mặt nghiệp vụ, vừa thể hiện tín hiệu hữu ích khi được đánh giá trong cùng pipeline và cùng hệ thống metric.

Kết quả cuối cùng của giai đoạn này là bộ feature được giữ lại để huấn luyện ba mô hình Logistic Regression, Random Forest và HistGradientBoosting. Sau khi so sánh trên tập Validation, HistGradientBoosting tiếp tục được lựa chọn làm mô hình ứng viên cho bước tối ưu ngưỡng ở Notebook 06.1.

4.4. Phân tích kết quả dự báo và gợi ý hành động

Trong Notebook 06, ngưỡng 0,5 chỉ được sử dụng như một ngưỡng tạm thời để so sánh Logistic Regression, Random Forest và HistGradientBoosting trên cùng tập Validation. Sau khi HistGradientBoosting được lựa chọn, quá trình tối ưu ngưỡng dự đoán được thực hiện riêng tại Notebook 06.1.

Notebook 06.1 sử dụng cùng cách chia dữ liệu với Notebook 06 theo tỷ lệ xấp xỉ **64% Train – 16% Validation – 20% Test**. Các bước tiền xử lý như loại cột hằng số, tạo lại hai interaction phụ thuộc phân vị, xử lý missing và one-hot encoding đều được xác định hoặc fit từ Train rồi áp dụng cố định cho Validation và Test.

Trong bài toán tín dụng, nhóm xem **False Negative (FN)** là trường hợp cần đặc biệt hạn chế. Đây là tình huống khách hàng thực tế thuộc nhóm có nguy cơ nợ xấu nhưng mô hình lại dự đoán là khách hàng trả được nợ. Vì mục tiêu nghiệp vụ cần ưu tiên Recall nhiều hơn Precision, nhóm sử dụng **F2-score** làm tiêu chí chính để lựa chọn ngưỡng.

F2-score được tính theo công thức:

$$F_2 = \frac{5 \times Precision \times Recall}{4 \times Precision + Recall}$$

Trên tập Validation, nhóm thử **91 ngưỡng từ 0,05 đến 0,95 với bước 0,01**. Tại mỗi ngưỡng, xác suất dự đoán được chuyển thành nhãn và tính Precision, Recall, F1-score và F2-score.

Các ngưỡng có F2-score cao nhất được trình bày trong bảng sau:

Ngưỡng	Precision	Recall	F1-score	F2-score
0,49	0,1795	0,6906	0,2850	0,4401
0,50	0,1829	0,6765	0,2879	0,4393
0,53	0,1940	0,6424	0,2980	0,4393
0,48	0,1757	0,7005	0,2809	0,4385
0,51	0,1860	0,6629	0,2905	0,4382
0,54	0,1976	0,6286	0,3006	0,4376
0,47	0,1724	0,7106	0,2775	0,4374
0,52	0,1890	0,6503	0,2929	0,4370
0,45	0,1668	0,7343	0,2718	0,4369
0,46	0,1693	0,7214	0,2742	0,4366

Kết quả cho thấy ngưỡng **0,49** đạt F2-score cao nhất trên Validation với **F2-score = 0,4401**, Precision = 0,1795, Recall = 0,6906 và F1-score = 0,2850. So với ngưỡng mặc định 0,50, ngưỡng 0,49 làm Recall tăng từ 0,6765 lên 0,6906, trong khi Precision giảm nhẹ từ 0,1829 xuống 0,1795. Đây là mức đánh đổi phù hợp với mục tiêu ưu tiên phát hiện khách hàng có nguy cơ nợ xấu.

Tập Test không tham gia vào quá trình lựa chọn ngưỡng. Chỉ sau khi ngưỡng đã được cố định từ Validation, mô hình mới được đánh giá cuối cùng trên Test.

Kết quả ROC-AUC và PR-AUC trên Test như sau:

Chỉ số	Giá trị
ROC-AUC	0,7757
PR-AUC	0,2706

Hai chỉ số này được tính trực tiếp từ xác suất dự đoán nên không phụ thuộc vào ngưỡng phân loại. Kết quả cho thấy HistGradientBoosting vẫn duy trì khả năng phân biệt và xếp hạng rủi ro tương đối ổn định trên tập Test.

Kết quả so sánh giữa ngưỡng mặc định 0,5 và ngưỡng được lựa chọn 0,49 được trình bày trong bảng sau:

Thiết lập	Ngưỡng	Precision	Recall	F1-score	F2-score
Ngưỡng mặc định	0,50	0,1827	0,6829	0,2883	0,4413
Ngưỡng được chọn	0,49	0,1793	0,6964	0,2852	0,4417

So với ngưỡng 0,5, ngưỡng 0,49 làm Recall tăng từ 0,6829 lên 0,6964, tương ứng tăng khoảng **1,35 điểm phần trăm**. F2-score cũng tăng nhẹ từ 0,4413 lên 0,4417. Đổi lại, Precision giảm từ 0,1827 xuống 0,1793 và F1-score giảm nhẹ từ 0,2883 xuống 0,2852.

Kết quả này phù hợp với mục tiêu lựa chọn ngưỡng bằng F2-score: mô hình chấp nhận tăng một phần số cảnh báo nhằm để phát hiện được nhiều khách hàng nợ xấu hơn.

Ma trận nhầm lẫn trên tập Test tại ngưỡng 0,49 cho kết quả:

Thực tế / Dự đoán	Trả được nợ	Nợ xấu
Trả được nợ	40.498	15.787
Nợ xấu	1.504	3.450

Trong đó:

- **40.498 True Negative (TN):** khách hàng thực tế trả được nợ và được dự đoán đúng.
- **15.787 False Positive (FP):** khách hàng trả được nợ nhưng bị cảnh báo là nợ xấu.
- **1.504 False Negative (FN):** khách hàng thực tế nợ xấu nhưng bị mô hình bỏ sót.
- **3.450 True Positive (TP):** khách hàng nợ xấu được mô hình phát hiện đúng.

Mô hình phát hiện đúng **3.450 trên tổng số 4.954 khách hàng nợ xấu**, tương ứng Recall 0,6964. Như vậy, khoảng 69,64% khách hàng nợ xấu trong tập Test được phát hiện, trong khi 1.504 trường hợp vẫn bị bỏ sót.

Tuy nhiên, Precision chỉ đạt 0,1793, nghĩa là trong số các hồ sơ bị cảnh báo rủi ro, tỷ lệ thực sự thuộc nhóm nợ xấu còn thấp. Đây là hệ quả của việc ưu tiên Recall trong bối cảnh dữ liệu mất cân bằng và cần được cân nhắc khi triển khai thực tế.

Ngưỡng 0,49 được lựa chọn vì đạt F2-score cao nhất trên Validation, nhưng không được xem là ngưỡng tối ưu tuyệt đối cho mọi chính sách tín dụng. Trong thực tế, tổ chức có thể

điều chỉnh threshold dựa trên chi phí của False Negative, False Positive và khẩu vị rủi ro tại từng thời điểm.

Mô hình không được thiết kế để tự động từ chối hồ sơ vay. Kết quả dự đoán nên được sử dụng như một tín hiệu hỗ trợ sàng lọc và ưu tiên thẩm định. Những hồ sơ có xác suất rủi ro cao có thể được chuyển sang bước kiểm tra chi tiết hơn trước khi đưa ra quyết định tín dụng cuối cùng.

Sau quá trình tối ưu ngưỡng và đánh giá cuối, mô hình HistGradientBoosting cùng threshold và metadata preprocessing được lưu tại:

models/hist_gradient_boosting_threshold_optimized.pkl

Các metric và cấu hình đánh giá được lưu tại:

models/hist_gradient_boosting_threshold_optimized_metrics.json

Hai artifact này được sử dụng trong thư mục app để xây dựng **FastAPI** cho chức năng dự đoán và **Streamlit** cho giao diện tương tác với người dùng.

Trong quá trình vận hành, cần theo dõi định kỳ ROC-AUC, PR-AUC, Precision, Recall, F1-score, F2-score và tỷ lệ nợ xấu thực tế. Khi phân bố khách hàng hoặc điều kiện kinh tế thay đổi đáng kể, mô hình cần được đánh giá lại và huấn luyện lại nếu cần thiết.

Chương 5. Quản trị dự án và phối hợp nhóm

5.1. Phân chia vai trò và nhiệm vụ (Team Roles & Responsibilities)

5.1.1. Project Manager

Nguyễn Thành Hưng – Trưởng nhóm, phụ trách lập kế hoạch, điều phối tiến độ, kiểm tra sự liên kết giữa các sản phẩm và quản lý quy trình làm việc chung trên GitHub.

5.1.2. Data Engineer

Đặng Trịnh Qui Anh và **Phạm Đức Thắng** – phụ trách tổ chức dữ liệu, thiết kế cơ sở dữ liệu PostgreSQL, import dữ liệu, tạo chỉ mục, tổng hợp bảng phụ và xây dựng quy trình join dữ liệu.

5.1.3. Data Analyst

Đỗ Trọng Thái – phụ trách làm sạch dữ liệu, phân tích khám phá dữ liệu, trực quan hóa insight và tổng hợp kết quả phân tích phục vụ báo cáo.

5.1.4. AI/Machine Learning Researcher

Nguyễn Bá Gia Huy – phụ trách chuẩn bị dữ liệu cho mô hình, huấn luyện và đánh giá các thuật toán phân loại, lựa chọn mô hình cuối cùng và xác định ngưỡng dự đoán.

5.2. Kế hoạch và tiến độ triển khai

Nhóm áp dụng cách làm việc theo từng giai đoạn ngắn, dựa trên tinh thần Agile/Scrum. Mỗi giai đoạn được chia thành các đầu việc cụ thể, giao cho thành viên phụ trách và kiểm tra lại trước khi hợp nhất vào sản phẩm chung.

5.2.1. Kế hoạch theo giai đoạn

Giai đoạn	Công việc chính	Thành viên phụ trách
Khảo sát bài toán và dữ liệu	Business Understanding, khảo sát cấu trúc dataset, xác định khóa nối	Nguyễn Thành Hưng
Tổ chức cơ sở dữ liệu	Thiết kế bảng PostgreSQL, import dữ liệu, tạo index, tổng hợp và join các bảng	Đặng Trịnh Qui Anh, Phạm Đức Thắng
Làm sạch và phân tích dữ liệu	Xử lý missing value, outlier, dữ liệu sai logic, thực hiện EDA và trực quan hóa insight	Đỗ Trọng Thái
Xây dựng đặc trưng	Tạo các feature về tài chính, lịch sử tín dụng và biến tương tác	Đặng Trịnh Qui Anh
Huấn luyện mô hình	Chuẩn bị dữ liệu, so sánh mô hình, chọn ngưỡng và lưu mô hình	Nguyễn Bá Gia Huy
Hoàn thiện sản phẩm	Tổng hợp báo cáo, slide và chuẩn bị hướng triển khai ứng dụng	Cả nhóm

5.2.2. Theo dõi công việc

Nhóm theo dõi tiến độ qua bảng công việc chung và GitHub. Mỗi đầu việc được cập nhật theo các trạng thái: **To Do**, **In Progress**, **Review** và **Done**. Với các phân mã nguồn và

notebook, thành viên làm việc trên nhánh riêng, sau đó tạo Pull Request để các thành viên khác kiểm tra trước khi hợp nhất vào nhánh chính. Cách làm này giúp nhóm biết rõ công việc nào đang thực hiện, công việc nào cần rà soát và hạn chế xung đột khi cùng làm trên một dự án.

ID	Task	Giai đoạn	Ngày	Người phụ trách	Trạng thái	Ngày HT	Ưu tiên
T01	Notebook 01 - Tìm hiểu dữ liệu (data understanding)	1. Data Engineering	03/07 - 08/07/2026	PS47270 Nguyễn Thành Hưng	Done - Hoàn thành	03/07/2026	Cao
T02	SQL 01 - Tạo bảng (create tables)	1. Data Engineering	03/07 - 08/07/2026	PS48172 Phạm Đức Thắng	Done - Hoàn thành	03/07/2026	Cao
T03	SQL 02 - Import dữ liệu vào PostgreSQL	1. Data Engineering	03/07 - 08/07/2026	PS48165 Đặng Trinh Qui Anh	Done - Hoàn thành	04/07/2026	Cao
T04	Notebook 02 - Pipeline PostgreSQL (kết nối, load)	1. Data Engineering	03/07 - 08/07/2026	PS48224 Nguyễn Bá Gia Huy	Done - Hoàn thành	13/07/2026	Trung b
T05	SQL 03 - Tạo views	1. Data Engineering	03/07 - 08/07/2026	PS47270 Nguyễn Thành Hưng	Done - Hoàn thành	12/07/2026	Trung b
T06	SQL 04 - Aggregation bằng phụ (bureau, previous_application...)	1. Data Engineering	03/07 - 08/07/2026	PS48224 Nguyễn Bá Gia Huy	Done - Hoàn thành	13/07/2026	Trung b
T07	SQL 05 - Tạo indexes tối ưu truy vấn	1. Data Engineering	03/07 - 08/07/2026	PS48172 Phạm Đức Thắng	Done - Hoàn thành	13/07/2026	Trung b
T08	Notebook 03 - Làm sạch dữ liệu (data cleaning)	1. Data Engineering	03/07 - 08/07/2026	PS47694 Đỗ Trọng Thái	Done - Hoàn thành	14/07/2026	Cao
T09	Notebook 04 - EDA & trực quan hóa	2. Data Science	09/07 - 12/07/2026	PS48224 Nguyễn Bá Gia Huy	Done - Hoàn thành	13/07/2026	Cao
T10	Notebook 05 - Feature engineering	2. Data Science	09/07 - 12/07/2026	PS48172 Phạm Đức Thắng	Done - Hoàn thành	13/07/2026	Cao
T11	Notebook 06 - Huấn luyện & đánh giá model	3. Modeling & Product	13/07 - 20/07/2026	PS48172 Phạm Đức Thắng	Done - Hoàn thành	15/07/2026	Cao
T12	[GÓP VÀO T11] Lưu model.pkl + scaler.pkl	3. Modeling & Product	13/07 - 20/07/2026	PS48172 Phạm Đức Thắng	Done - Hoàn thành	15/07/2026	Thấp
T19	README.md + requirements.txt	1. Data Engineering	03/07 - 20/07/2026	PS47270 Nguyễn Thành Hưng	Done - Hoàn thành	15/07/2026	Trung b
T30	Notebook 01 - bổ sung theo checklist mới của thầy	5. Làm lại sau reset	21/07 - 31/07/2026	PS47270 Nguyễn Thành Hưng	Done - Hoàn thành	25/07/2026	Cao
T31	Notebook 02 - Database Organization (làm lại)	5. Làm lại sau reset	21/07 - 12/08/2026	PS47270 Nguyễn Thành Hưng	Done - Hoàn thành	12/08/2026	Cao
T32	Bộ SQL 11 file (làm lại toàn bộ)	5. Làm lại sau reset	21/07 - 14/08/2026	PS47270 Nguyễn Thành Hưng	Done - Hoàn thành	14/08/2026	Cao
T33	Tài liệu hướng dẫn giảng viên NB01/NB02/NB03	5. Làm lại sau reset	21/07 - 31/07/2026	PS47270 Nguyễn Thành Hưng	Done - Hoàn thành	22/07/2026	Trung b
T34	Notebook 03 - Data Cleaning (làm lại)	5. Làm lại sau reset	21/07 - 14/08/2026	PS47694 Đỗ Trọng Thái	Done - Hoàn thành	14/08/2026	Cao
T35	Notebook 04 - EDA & Visualization (làm lại)	6. EDA & Feature	28/07 - 13/08/2026	PS48224 Nguyễn Bá Gia Huy	Done - Hoàn thành	13/08/2026	Cao
T36	Notebook 05 - Feature Engineering (làm lại)	6. EDA & Feature	31/07 - 14/08/2026	PS48165 Đặng Trinh Qui Anh	Done - Hoàn thành	14/08/2026	Cao
T37	Notebook 06 - Modeling & Evaluation (làm lại)	7. Modeling & Optimization	06/08 - 14/08/2026	PS48224 Nguyễn Bá Gia Huy	Done - Hoàn thành	14/08/2026	Cao
T38	Notebook 06.1 - Threshold Optimization	7. Modeling & Optimization	14/08/2026	PS47270 Nguyễn Thành Hưng	Done - Hoàn thành	14/08/2026	Cao
T39	Cập nhật whitepaper 6 chương theo pipeline mới	8. Documentation	14/08/2026 - hiện tại	PS47270 Nguyễn Thành Hưng	Done - Hoàn thành		Cao
T40	Notebook 07 - Prediction Demo	9. Product	Sau khi chốt báo cáo	PS47270 Nguyễn Thành Hưng	In Progress - Đang thực hiện		Cao
T41	Hoàn thiện slide báo vệ 16 trang	8. Documentation	Sau khi chốt báo cáo	Cả nhóm (mỗi người notebook của mình)	In Progress - Đang thực hiện		Cao
T42	Streamlit và dashboard tương tác	9. Product	Sau NB07	PS47270 Nguyễn Thành Hưng	In Progress - Đang thực hiện		Cao

Hình 5.1 Bảng phân công công việc

5.2.3. Báo cáo tiến độ và xử lý khó khăn

Tiến độ được tổng hợp định kỳ từ bảng công việc chung, lịch sử commit và Pull Request. Khi phát sinh khó khăn như dữ liệu lớn, lỗi kiểu dữ liệu, sai lệch sau join hoặc kết quả mô hình chưa đạt kỳ vọng, thành viên phụ trách ghi nhận nguyên nhân, trao đổi bằng bằng chứng từ output và điều chỉnh trong nhánh riêng. Cách làm này giúp nhóm xử lý vấn đề minh bạch và không che giấu các thử nghiệm chưa cải thiện metric.

5.3. Quản lý mã nguồn và phiên bản trên Git/GitHub

Nhóm sử dụng Git và GitHub để quản lý mã nguồn, lưu vết thay đổi và phối hợp công việc. Quy trình này giúp các thành viên có thể phát triển những phần việc khác nhau mà vẫn hạn chế xung đột, đồng thời bảo đảm phiên bản trên nhánh chính luôn là phiên bản ổn định



Uncommitted Changes (7)				
	fix/hung-capnhat-laiieu-rp-14082026		main	
	fix/hung-fix-NB06-13082026		origin	
	fix/hung-fix-NB05-13082026		origin/HEAD	
	Merge pull request #118 from thanhhung9720001...			
	Merge pull request #117 from thanhhung9720001-collab/fix/hung-fix-NB05-13082026			
	fix/hung-fix-NB05-13082026		origin	
	Merge pull request #116 from thanhhung9720001-collab/fix/hung-fix-NBs-12082026			
	fix/hung-fix-NBs-12082026		origin	
	Merge pull request #115 from thanhhung9720001-collab/fix/fix-NBs-by-Hung-09082026			
	fix/fix-NBs-by-Hung-09082026		origin	
	Merge pull request #114 from thanhhung9720001-collab/fix/fix-NBs-by-Hung-09082026			
	fix: Hung fix NB00, NB01, NB02 ngày 12082026			
	Merge pull request #113 from thanhhung9720001-collab/docs/hung-capnhat-tailieu-12082026			
	docs/hung-capnhat-tailieu-12082026		origin	
	Merge pull request #112 from thanhhung9720001-collab/docs/Hung-file-tailieu-11082026			
	docs/Hung-file-tailieu-11082026		origin	
	Fix: Đăng fix NB01, NB02			
	Merge pull request #111 from thanhhung9720001-collab/docs/thang-quanh-powerpoint			
	Merge pull request #110 from thanhhung9720001-collab/feature/t11-huy-modeling			
	docs: cap nhat context ca nhan			
	docs: cap nhat slide bao cao powerpoint			
	feat: huy hoan thien NB06 muc VII.3 den IX va don so lieu cu			
	Merge pull request #108 from thanhhung9720001-collab/fix/thai-chuong2-so-lieu-nb02-nb03-07082026			
	fix: sua so lieu Chuong 2 cho khop voi NB02 va NB03 that			
	feat/hung-lam-NB06copy		origin	
	Merge pull request #107 from thanhhung9720001-collab/docs/thai-chuong3-eda-featureengineerin...			
	docs: viet Chuong 3 EDA va Feature Engineering (NB04, NB05)			
	Merge pull request #106 from thanhhung9720001-collab/feat/hung-feat-NB06-06082026			

Hình 5.2 Sơ đồ git quản lý mã nguồn chung

5.3.1. Chiến lược phân nhánh

Nhánh main được sử dụng để lưu phiên bản ổn định của dự án và không được chỉnh sửa hoặc push trực tiếp. Mỗi nhiệm vụ được thực hiện trên một nhánh riêng theo quy ước như feature/<ten-tinh-nang>, fix/<ten-sua-loi> hoặc docs/<ten-tai-lieu>. Ví dụ, các công việc về làm sạch dữ liệu, Feature Engineering, mô hình hóa và cập nhật báo cáo được tách thành các nhánh độc lập trước khi đưa vào main.

5.3.2. Quy trình Commit và Pull Request

Trước khi bắt đầu công việc, thành viên cập nhật mã nguồn mới nhất và kiểm tra nhánh hiện tại. Sau khi hoàn thành một phần việc, thành viên commit với nội dung ngắn gọn, mô tả rõ thay đổi đã thực hiện. Các thay đổi được push lên nhánh riêng và tạo Pull Request để các thành viên kiểm tra nội dung, kết quả chạy notebook và nguy cơ xung đột trước khi hợp nhất.

Nhánh main được bảo vệ bằng GitHub Ruleset. Quy trình này ngăn việc push trực tiếp, yêu cầu thay đổi đi qua Pull Request và chỉ cho phép trưởng nhóm hợp nhất sau khi kiểm tra. Nhờ đó, nhóm có thể kiểm soát chất lượng mã nguồn và dễ dàng truy vết lịch sử phát triển của dự án.

5.3.3. Tài liệu hướng dẫn dự án

File README.md đóng vai trò là tài liệu hướng dẫn chính của dự án. Tài liệu này mô tả mục tiêu dự án, cấu trúc thư mục, các thư viện cần thiết và hướng dẫn chạy các thành phần của hệ thống. Bên cạnh đó, các file notebook được tổ chức theo thứ tự của pipeline, từ Business Understanding, Data Understanding, tổ chức cơ sở dữ liệu, làm sạch dữ liệu, EDA, Feature Engineering đến huấn luyện mô hình. Cách tổ chức này giúp thành viên mới hoặc người chấm có thể theo dõi và chạy lại quy trình một cách thuận tiện.

5.4. Quy tắc cộng tác và đạo đức nghề nghiệp

5.4.1. Quản lý sản phẩm chung

Nhóm thống nhất tổ chức mã nguồn, notebook, tài liệu, mô hình và báo cáo theo cấu trúc thư mục của dự án. Trước khi nộp, toàn bộ sản phẩm được kiểm tra để bảo đảm có đầy đủ mã nguồn, tài liệu hướng dẫn, báo cáo, slide và các thành phần cần thiết để chạy lại dự án. Sản phẩm cuối cùng được đóng gói theo đúng định dạng yêu cầu và nộp đúng thời hạn

5.4.2. Minh bạch về nguồn tham khảo

Nhóm cam kết tôn trọng tính trung thực học thuật. Các tài liệu, ý tưởng, đoạn mã hoặc phương pháp tham khảo từ nguồn bên ngoài được ghi nhận rõ ràng trong notebook, README hoặc tài liệu báo cáo khi cần thiết. Nhóm không sử dụng kết quả của nguồn khác như kết quả do mình tự thực hiện, đồng thời luôn đối chiếu lại mã và số liệu với dữ liệu thực tế của dự án.

5.4.3. Kỷ luật và tinh thần hợp tác

Các thành viên tuân thủ quy trình làm việc chung, làm việc trên nhánh riêng và không tự ý thay đổi phần việc của thành viên khác. Khi có ý kiến phản biện hoặc phát hiện lỗi, nhóm trao đổi dựa trên kết quả thực tế của dữ liệu, notebook và mã nguồn. Mỗi thành viên có trách nhiệm hỗ trợ nhau trong các vấn đề kỹ thuật, chủ động cập nhật tiến độ và cùng chịu trách nhiệm về chất lượng sản phẩm cuối cùng.

Chương 6. Tổng kết và triển khai dự án

6.1. Hướng dẫn đóng gói và chạy dự án

Khi nộp bài, nhóm đóng gói dự án theo cấu trúc thư mục rõ ràng để người chấm có thể dễ dàng kiểm tra mã nguồn, notebook, tài liệu, mô hình và phần triển khai ứng dụng. Do dữ liệu gốc của bộ Home Credit Default Risk có dung lượng lớn, nhóm có thể không đưa toàn bộ dữ liệu raw vào file nén mà thay bằng đường dẫn hoặc hướng dẫn tải dữ liệu từ Kaggle.

Cấu trúc thư mục dự án được tổ chức như sau:

credit-risk-classifier/

└─ notebooks/ # Các notebook theo thứ tự của pipeline

└─ sql/ # Script tạo bảng, import, index, summary, join và kiểm thử SQL

└─ data/ # Dữ liệu raw hoặc hướng dẫn/link tải dữ liệu

└─ models/ # Mô hình đã huấn luyện và metadata/preprocessing

└─ docs/ # Đề bài, tài liệu tham khảo và tài liệu dự án

└─ reports/ # Hình ảnh, báo cáo Word và slide trình bày

└─ app/ # Mã nguồn FastAPI và Streamlit phục vụ dự đoán

└─ requirements.txt

└─ README.md

Để chạy lại dự án trên máy cục bộ, người dùng cần cài đặt Python, PostgreSQL và các thư viện được khai báo trong file requirements.txt bằng lệnh:

```
pip install -r requirements.txt
```

Tiếp theo, cần tạo file .env tại thư mục gốc của dự án để khai báo thông tin kết nối PostgreSQL, gồm:

DB_HOST

DB_PORT

DB_NAME

DB_USER

DB_PASSWORD

File .env chứa thông tin kết nối và mật khẩu nên không được đưa lên GitHub hoặc đóng gói công khai.

Các notebook được thực hiện theo thứ tự:

00_business_understanding.ipynb

→ 01_data_understanding.ipynb

→ 02_database_organization.ipynb

→ 03_data_cleaning.ipynb

→ 04_eda_visualization.ipynb

→ 05_feature_engineering.ipynb

→ 06_modeling_evaluation.ipynb

→ 06_1_threshold_optimization.ipynb

Trước khi thực hiện các notebook phụ thuộc PostgreSQL, cần tạo database và chạy các script SQL theo đúng thứ tự của dự án để tạo bảng raw, import dữ liệu, xây dựng index, các bảng summary và bảng dữ liệu tổng hợp. Từ giai đoạn Data Cleaning trở đi, các notebook sử dụng dữ liệu đã được tổ chức và lưu trong PostgreSQL, bao gồm các bảng trung gian như application_flat, application_flat_cleaned và application_features.

Notebook 06 thực hiện huấn luyện và so sánh Logistic Regression, Random Forest và HistGradientBoosting. Dữ liệu được chia theo tỷ lệ xấp xỉ **64% Train – 16% Validation – 20% Test**. Tập Train được dùng để học preprocessing và huấn luyện mô hình, Validation được dùng để lựa chọn mô hình, còn Test được giữ riêng cho bước đánh giá cuối cùng.

Sau khi HistGradientBoosting được lựa chọn, Notebook 06.1 tiếp tục thử các ngưỡng dự đoán trên Validation và lựa chọn ngưỡng theo **F2-score**. Ngưỡng cuối được lựa chọn là **0,49**, sau đó mô hình được đánh giá một lần trên tập Test.

Sau khi chạy hoàn chỉnh Notebook 06.1, thư mục models/ cần có hai artifact chính:

hist_gradient_boosting_threshold_optimized.pkl

hist_gradient_boosting_threshold_optimized_metrics.json

File hist_gradient_boosting_threshold_optimized.pkl chứa mô hình HistGradientBoosting, ngưỡng dự đoán cuối 0,49, danh sách feature sau preprocessing và các thành phần cần thiết để tái tạo dữ liệu đầu vào, bao gồm thông tin xử lý missing, OneHotEncoder, các cột hằng số, quy tắc xử lý dữ liệu lịch sử và metadata của hai interaction feature.

File `hist_gradient_boosting_threshold_optimized_metrics.json` lưu các thông tin đánh giá chính của mô hình, gồm ngưỡng được lựa chọn, các metric trên Validation và Test, ROC-AUC, PR-AUC, Precision, Recall, F1-score, F2-score và ma trận nhầm lẫn cuối cùng.

Các artifact trong thư mục `models/` được sử dụng bởi phần triển khai trong thư mục `app/`. FastAPI chịu trách nhiệm cung cấp các endpoint phục vụ dự đoán, trong khi Streamlit cung cấp giao diện để người dùng nhập hoặc lựa chọn thông tin khách hàng và xem kết quả dự báo rủi ro.

Nếu các file mô hình không được đưa kèm trong repository do cấu hình `.gitignore`, môi trường triển khai cần nhận đúng các artifact đã tạo hoặc chạy lại Notebook 06.1 để sinh lại chúng trước khi khởi động FastAPI và Streamlit.

Đối với Kaggle hoặc Google Colab, các notebook đầu như Data Understanding có thể chạy trực tiếp khi dataset được gắn vào môi trường. Tuy nhiên, từ giai đoạn tổ chức dữ liệu trở đi, dự án phụ thuộc vào PostgreSQL và các bảng trung gian đã được xây dựng trong database. Vì vậy, chạy dự án trên máy cục bộ hoặc môi trường có PostgreSQL được cấu hình đầy đủ là phương án tái lập toàn bộ pipeline thuận tiện và ổn định hơn.

Khi chuyển sang một môi trường khác, cần bảo đảm ba điều kiện chính: cấu hình kết nối PostgreSQL chính xác, các bảng dữ liệu trung gian đã được tạo đầy đủ và preprocessing khi dự đoán phải sử dụng đúng artifact đã lưu cùng mô hình. Điều này giúp thứ tự feature và cách xử lý dữ liệu khi inference nhất quán với giai đoạn huấn luyện.

6.2. Kết luận

Dự án đã xây dựng hoàn chỉnh pipeline phân loại và dự báo rủi ro tín dụng trên bộ dữ liệu Home Credit Default Risk. Dữ liệu ban đầu được tổ chức trên PostgreSQL từ các bảng raw, sau đó tổng hợp về mức khách hàng và kết hợp thành bảng `application_flat`. Các bước kiểm tra dữ liệu được thực hiện nhằm bảo đảm tính duy nhất của `sk_id_curr`, phân bố của biến

mục tiêu và ý nghĩa của các trường dữ liệu thiếu trước khi chuyển sang các giai đoạn tiếp theo.

Sau bước Data Cleaning, dữ liệu được làm sạch về mặt logic, xử lý giá trị thiếu và giới hạn các giá trị ngoại lệ. Kết quả cuối của giai đoạn này là bảng `application_flat_cleaned` với **306.195 dòng × 188 cột**, được sử dụng làm đầu vào cho EDA và Feature Engineering.

Trong giai đoạn EDA, nhóm phân tích nhiều nhóm biến liên quan đến đặc điểm khách hàng, khoản vay, thu nhập, điểm đánh giá ngoài, lịch sử tín dụng và hành vi thanh toán. Kết quả cho thấy các yếu tố như điểm đánh giá ngoài, tỷ lệ khoản vay trên giá trị tài sản, độ tuổi, lịch sử chậm trả và dư nợ còn lại có khả năng cung cấp tín hiệu hữu ích cho việc phân biệt khách hàng có nguy cơ nợ xấu.

Notebook 05 tiếp tục xây dựng các feature nghiệp vụ mới từ dữ liệu đã làm sạch. Kết quả tạo ra bảng `application_features` gồm **306.195 dòng × 234 cột**. Các feature được kiểm chứng qua nhiều vòng nhằm đánh giá mức đóng góp thực tế thay vì chỉ dựa trên ý nghĩa nghiệp vụ.

Trong Notebook 06, dữ liệu được chia theo tỷ lệ xấp xỉ **64% Train – 16% Validation – 20% Test**. Tập Train được sử dụng để học preprocessing và huấn luyện mô hình, tập Validation được dùng để so sánh và lựa chọn mô hình, còn Test được giữ riêng cho bước đánh giá cuối cùng.

Ba mô hình Logistic Regression, Random Forest và HistGradientBoosting được huấn luyện và đánh giá trên cùng điều kiện tại ngưỡng tạm thời 0,5. Kết quả trên tập Validation như sau:

Mô hình	ROC-AUC	PR-AUC	Precision	Recall	F1-score
Logistic Regression	0,7667	0,2463	0,1729	0,6881	0,2763
Random Forest	0,7601	0,2438	0,1877	0,5985	0,2858
HistGradientBoosting	0,7745	0,2598	0,1829	0,6765	0,2879

HistGradientBoosting đạt kết quả tốt nhất về ROC-AUC, PR-AUC và F1-score nên được lựa chọn làm mô hình cuối cùng để tiếp tục tối ưu ngưỡng dự đoán.

Tại Notebook 06.1, nhóm thử **91 ngưỡng từ 0,05 đến 0,95** trên tập Validation và sử dụng **F2-score** làm tiêu chí lựa chọn. F2-score được ưu tiên vì bài toán cần hạn chế False Negative và chú trọng khả năng phát hiện khách hàng có nguy cơ nợ xấu.

Ngưỡng được lựa chọn là **0,49**, đạt trên Validation:

- Precision: **0,1795**
- Recall: **0,6906**
- F1-score: **0,2850**
- F2-score: **0,4401**

Sau khi ngưỡng được cố định, mô hình được đánh giá cuối cùng trên tập Test. Kết quả đạt:

Chỉ số	ROC-AUC	PR-AUC	Precision	Recall	F1-score	F2-score
Giá trị	0,7757	0,2706	0,1793	0,6964	0,2852	0,4417

So với ngưỡng mặc định 0,5, ngưỡng 0,49 giúp Recall tăng từ 0,6829 lên 0,6964 và F2-score tăng từ 0,4413 lên 0,4417, trong khi Precision giảm nhẹ từ 0,1827 xuống 0,1793. Kết quả này phù hợp với định hướng nghiệp vụ của dự án là ưu tiên phát hiện thêm khách hàng có nguy cơ nợ xấu.

Ma trận nhầm lẫn tại ngưỡng 0,49 cho thấy mô hình dự đoán đúng **40.498** khách hàng trả được nợ và phát hiện đúng **3.450** khách hàng nợ xấu. Đồng thời, có **15.787** khách hàng trả được nợ bị cảnh báo nhầm và **1.504** khách hàng nợ xấu bị bỏ sót. Như vậy, mô hình phát hiện được khoảng **69,64%** số khách hàng nợ xấu trong tập Test.

Kết quả cuối cho thấy mô hình có khả năng hỗ trợ sàng lọc rủi ro, nhưng Precision còn thấp do dữ liệu mất cân bằng và định hướng ưu tiên Recall. Vì vậy, mô hình không nên được sử dụng như cơ chế tự động từ chối khoản vay mà phù hợp hơn với vai trò **hỗ trợ sàng lọc và ưu tiên hồ sơ cần thẩm định kỹ hơn**.

Sau khi hoàn thành quá trình huấn luyện và tối ưu ngưỡng, mô hình HistGradientBoosting, threshold và các artifact preprocessing được lưu trong thư mục models/. Các artifact này được sử dụng để triển khai **FastAPI** và **Streamlit** trong thư mục app, phục vụ dự đoán và tương tác với người dùng.

Nhìn chung, dự án đã hoàn thành đầy đủ chuỗi xử lý từ tổ chức dữ liệu, làm sạch, EDA, Feature Engineering, huấn luyện mô hình, lựa chọn mô hình, tối ưu threshold đến chuẩn bị artifact cho triển khai. Kết quả đạt được cho thấy pipeline có khả năng tái lập và có thể tiếp tục mở rộng khi có thêm dữ liệu thực tế hoặc yêu cầu nghiệp vụ mới.

6.3. Hướng phát triển

Trong tương lai, dự án có thể được mở rộng theo cả hướng cải thiện mô hình, bổ sung dữ liệu và hoàn thiện phần triển khai ứng dụng.

Trước hết, nhóm có thể thử nghiệm thêm các thuật toán phân loại khác như XGBoost, LightGBM hoặc CatBoost để so sánh với HistGradientBoosting hiện tại. Bên cạnh đó, có thể thực hiện Hyperparameter Tuning cho mô hình đang sử dụng, chẳng hạn điều chỉnh `learning_rate`, `max_iter`, `max_leaf_nodes` hoặc `min_samples_leaf`, nhằm cải thiện khả năng phân biệt khách hàng có nguy cơ nợ xấu.

Về dữ liệu, dự án có thể bổ sung thêm các feature phản ánh hành vi tín dụng gần thời điểm hiện tại, lịch sử thanh toán, mức sử dụng hạn mức và các chỉ số tài chính có ý nghĩa nghiệp vụ. Những feature mới cần tiếp tục được kiểm chứng bằng cùng pipeline và các metric phù hợp thay vì chỉ giữ lại dựa trên giả thuyết nghiệp vụ.

Khi có thêm dữ liệu thực tế theo thời gian, mô hình nên được huấn luyện và đánh giá lại để thích nghi với sự thay đổi trong hành vi vay và trả nợ của khách hàng. Việc mở rộng dữ liệu cũng giúp kiểm tra khả năng tổng quát hóa của mô hình tốt hơn so với chỉ sử dụng bộ dữ liệu ban đầu.

Một hướng phát triển quan trọng khác là mở rộng phần triển khai trong thư mục app. FastAPI có thể được bổ sung thêm các endpoint phục vụ tra cứu, kiểm tra trạng thái hệ thống, lưu lịch sử dự đoán hoặc quản lý phiên bản mô hình. Streamlit có thể được mở rộng để hiển thị xác suất rủi ro, mức cảnh báo, các thông tin liên quan đến hồ sơ khách hàng và lịch sử dự đoán theo cách trực quan hơn.

Dự án cũng có thể bổ sung các phương pháp giải thích mô hình như SHAP hoặc các kỹ thuật giải thích cục bộ để giúp người dùng hiểu vì sao một hồ sơ được đánh giá có mức rủi ro cao hoặc thấp. Tuy nhiên, các kết quả giải thích cần được sử dụng như công cụ hỗ trợ và không được diễn giải như quan hệ nguyên nhân trực tiếp.

Sau khi triển khai, cần xây dựng cơ chế theo dõi định kỳ các chỉ số như ROC-AUC, PR-AUC, Precision, Recall, F1-score, F2-score và tỷ lệ nợ xấu thực tế. Đồng thời, nên theo dõi sự thay đổi của phân bố dữ liệu đầu vào để phát hiện **Data Drift** và theo dõi sự suy giảm hiệu suất dự báo để phát hiện **Model Drift**.

Nếu xuất hiện sự thay đổi đáng kể về dữ liệu hoặc hiệu suất mô hình, nhóm cần cập nhật dữ liệu, đánh giá lại preprocessing, huấn luyện lại mô hình và lựa chọn lại threshold trên tập Validation trước khi đưa phiên bản mới vào sử dụng.

Cuối cùng, để tiến gần hơn đến môi trường thực tế, dự án có thể bổ sung cơ chế quản lý phiên bản model, logging, kiểm thử API, xác thực dữ liệu đầu vào và container hóa bằng Docker. Các cải tiến này giúp hệ thống dễ triển khai, dễ bảo trì và có khả năng mở rộng tốt hơn khi số lượng yêu cầu dự đoán tăng lên.